

**CYBERGUN: A MODULAR,
MULTI-LAYERED FRAMEWORK FOR
REAL-TIME MALWARE DETECTION AND
AUTOMATED THREAT RESPONSE**



**Submitted by
Zahid Ullah**

**Supervised by
Dr. Ibrar Ahmad**

**BS Computer Science
Session 2021-25**

**Department of Computer Science
University of Peshawar**

Project Approval

This is to certify that this project is approved and recommended as a partial fulfillment for the degree “Bachelor of Computer Science” from the Department of Computer Science, University of Peshawar.

External Examiner

Name: _____

Signature: _____

Remarks: _____

Internal Examiner

Name: _____

Signature: _____

Remarks: _____

Chairman

Name: _____

Signature: _____

Remarks: _____

Acknowledgement

This thesis is the culmination of years of effort, and it would not have been possible without the guidance, support, and encouragement of many individuals. I would like to take this opportunity to express my sincere gratitude to all of them. First and foremost, I wish to express my deepest gratitude to my thesis supervisor, Dr. Ibrar Ahmad. Your invaluable guidance, persistent encouragement, and profound expertise were the bedrock of this research. Your insightful feedback challenged me to think critically, and your patience in our many discussions helped me to navigate the complexities of this project. Thank you for believing in the potential of this work and for pushing me to do my best.

Above all, I owe my deepest and most heartfelt thanks to my family. To my parents, Muhammad Zahid Shah and Jamala Bibi, your unwavering love, endless support, and constant sacrifice have been my foundation throughout this entire journey and, indeed, my entire life. To my friend Shahab Ullah, thank you for your incredible support, patience, understanding, and encouragement, especially during the most demanding periods of this research. This achievement is as much yours as it is mine. Finally, I dedicate this work to my mother, whose unparalleled sacrifices are behind each breath of my life, without whom none of this would have been possible.

Mr. Zahid Ullah

Abstract

The escalating sophistication and volume of malicious software present a significant challenge to modern digital security. Polymorphic and multi-stage threats can often circumvent traditional security solutions that rely on singular detection methodologies. This research addresses the critical need for a more holistic and integrated endpoint security system capable of countering diverse attack vectors.

This thesis presents CyberGun, a novel, modular framework designed for comprehensive, real-time malware detection and automated threat response. The framework is architecturally divided into three synergistic pillars: static analysis, dynamic behavioral monitoring, and network traffic analysis. Static analysis employs a computationally efficient layered approach, combining hash matching, YARA rules, and machine learning. Specifically, LightGBM models trained on the benchmark EMBER dataset are utilized for high-performance classification of various file types, including Portable Executable (PE), Executable and linkable Format (ELF), and Android Package (APK). Dynamic analysis implements an innovative behavioral funnel; it first uses machine learning models (Random Forest, XGBoost) trained on the CIC-MalMem-2022 dataset to identify memory anomalies, then deploys the Frida framework for deep API tracing of suspicious processes. These traces are subsequently classified by models trained on the MalAPI-2019 dataset. All detection mechanisms are interconnected via a decoupled signal bus architecture, which triggers an automated threat responder for immediate neutralization and quarantine of identified threats.

Experimental evaluation demonstrates the efficacy of the CyberGun framework. The constituent machine learning models achieve high accuracy on their respective benchmark test sets, and the integrated system shows a high detection rate with low false positives when tested against a curated set of real-world malware samples. The framework's design maintains efficient resource utilization, even during active monitoring. This research concludes that CyberGun's integrated, multi-layered, and machine learning driven architecture represents a robust and effective paradigm for modern endpoint security.

Table of Contents

1	Introduction	1
1.1	Malware Detection and Response Systems	2
1.2	Background and Motivation	2
1.3	Problem Description	3
1.4	Proposed Solution: CyberGun	4
1.5	Aims and Objectives	5
1.6	Key Contributions	6
1.7	Project Significance	7
1.8	Project Scope	9
1.9	Thesis Organization	10
2	Feasibility and Analysis	12
2.1	Existing Systems And their Drawbacks	12
2.2	Proposed System and its features	13
2.3	Feasibility Report of Proposed System	14
2.3.1	Technical Feasibility	14
2.3.2	Economic Feasibility	15
2.3.3	Social Feasibility	15
3	Requirement Specification	16
3.1	Functional Requirements	16
3.1.1	Multi-Layered Static Analysis	16
3.1.2	Real-Time System Monitoring	17
3.1.3	Network Threat Detection	18
3.1.4	Automated Threat Response & Management	19

3.2	Non-Functional Requirements	19
3.2.1	Performance	19
3.2.2	Reliability & Stability	20
3.2.3	Usability	20
3.2.4	Maintainability & Scalability	21
3.3	UI Requirements	21
3.3.1	Main Dashboard View	21
3.3.2	Other Views	22
3.3.3	Controls and Interactivity	22
3.4	User Interactions with the System through Use Case Diagram	23
3.4.1	System Boundary	24
3.4.2	Development Environment Requirements	25
3.4.3	Deployment Requirements	26
4	System Design and Architecture	28
4.1	Architecture Design	28
4.2	Data Persistence and Data Access Design	30
4.2.1	Data Persistence Design Details	30
4.2.2	Data Access Design Details	31
4.3	Component Design	33
4.3.1	System Workflow	33
4.3.2	System Workflow	36
4.3.3	User-Interface Design	39
5	Implementation and Testing	40
5.1	Development Environment Setup	40
5.1.1	Technology Stack	41
5.1.2	Repository Setup & Version Control	42
5.2	Implementation of Key Features	42
5.2.1	The Layered Static Analysis Engine: A Strategy for Efficiency	42
5.2.1.1	On-Demand Resource Management	44
5.2.1.2	Real-Time Threat Reporting	44

5.2.2	The Behavioral Analysis Funnel: From Anomaly to Verdict . .	45
5.2.3	The Automated Threat Response Mechanism: Closing the Loop	47
5.3	Testing	49
5.3.1	White Box Testing	50
5.3.2	Black Box Testing	50
5.3.3	Usability and User Testing	51
5.3.4	Testing Summary	51
6	Experimental Evaluation and Results	52
6.1	Experimental Setup	52
6.1.1	Hardware and Software Environment	52
6.1.2	Datasets	53
6.2	Performance Analysis	53
6.2.1	Resource Utilization	53
6.2.2	Static Scan Speed	54
6.3	Machine Learning Model Accuracy	55
6.3.1	Static Model Evaluation (MLDetector)	55
6.3.2	Dynamic System State Model Evaluation(SystemStateClassifier)	55
6.3.3	Dynamic API Sequence Model Evaluation (ApiSequenceClas- sifier)	56
6.4	End-to-End System Efficacy	56
7	Conclusion and Future Work	58
7.1	Summary of Contributions and Findings	58
7.2	Limitations of the Current Work	59
7.3	Future Work	60

List of Tables

6.1	Confusion Matrix for MLDetector by File Type	55
6.2	Classification Metrics for SystemStateClassifier	55
6.3	End-to-End System Performance	57

List of Figures

1.1	Isolated security models vs. integrated CyberGun architecture.	3
3.1	Use Case Diagram of the CyberGun System.	24
4.1	Architecture of the proposed system	29
4.2	Component Diagram of CyberGun showing core modules and their interactions.	32
4.3	An Activity Diagram illustrating system workflow.	35
4.4	Data Flow Diagram (DFD) - Level 0.	37
4.5	Data Flow Diagram (DFD) - Level 1.	38
5.1	Flowchart illustrating the logic of the <code>scan_directory_layered</code> method, highlighting the sequential layered scanning and early exit strategy. . . .	45
5.2	: Dynamic Analysis Funnel showing the progression from all system processes to high-confidence threat verdicts.	47
5.3	:UML Sequence Diagram for Threat Response.	49
6.1	CyberGun CPU and Memory Usage Over Time	54
6.2	Confusion matrix Logistic Random Forest	56
6.3	Threats First Detected by Engine Layer	57

List of Abbreviations

API	Application Programming Interface
APK	Android Package
APT	Advanced Persistent Threat
AV	Antivirus
C2	Command-and-Control
CNI	Critical National Infrastructure
CPU	Central Processing Unit
CSV	Comma-Separated Values
DDR4	Double Data Rate 4
DFD	Data Flow Diagram
DGA	Domain Generation Algorithm
DPI	Deep Packet Inspection
EDR	Endpoint Detection and Response
ELF	Executable and Linkable Format
EMBER	Endgame Malware Benchmark for Evaluation and Research
F1	F1-Score
FIPS	Federal Information Processing Standards
FN	False Negative
FP	False Positive
GUI	Graphical User Interface
IP	Internet Protocol
JSON	JavaScript Object Notation
LotL	Living-off-the-Land
ML	Machine Learning
NIST	National Institute of Standards and Technology
PE	Portable Executable

PDF	Portable Document Format
PID	Process Identifier
RAM	Random-Access Memory
SHA	Secure Hash Algorithm
TF-IDF	Term Frequency-Inverse Document Frequency
TN	True Negative
TP	True Positive
UI	User Interface
UML	Unified Modeling Language
USB	Universal Serial Bus

Chapter 1

Introduction

In today's digital world, malware has become one of the most persistent and adaptive threats we face. What once started as simple computer viruses that could be easily recognized and blocked has evolved into a highly sophisticated battlefield. Modern attackers now rely on advanced techniques and tools, creating everything from ransomware that locks entire organizations out of their systems, to spyware that silently watches users, to stealthy trojans and rootkits built to remain hidden for as long as possible.

A big reason behind this growing complexity is the use of polymorphic and metamorphic techniques. Polymorphic malware can change its code and structure each time it spreads, making traditional signature-based detection almost useless. Metamorphic malware goes even further by rewriting its own code entirely, changing both how it looks and how it behaves, while still carrying out the same malicious purpose.

Attackers have also shifted toward multi-stage and fileless approaches. A dropper file that looks harmless may quietly download the real payload later, or malicious code might run directly in memory without ever touching the disk — leaving behind little to no trace. These kinds of strategies are designed specifically to bypass older, static defenses.

Because of this, cybersecurity can no longer rely on one method of defense. The arms race between attackers and defenders has pushed us to rethink our strategies — moving from simple signature-based tools to intelligent, layered, and adaptive systems that combine different techniques and learn from new threats in real time.

1.1 Malware Detection and Response Systems

The project lies within the domain of applied cybersecurity, focusing on intelligent, real-time malware detection and automated threat response. CyberGun is a modular, multi-layered security framework that integrates static, dynamic, and network analysis, enhanced by machine learning, to provide proactive defense against modern cyber threats.

1.2 Background and Motivation

The contemporary digital landscape is characterized by an escalating and ever-evolving threat from malicious software, or malware. The era of simple, easily cataloged viruses has been superseded by a sophisticated threat landscape populated by polymorphic ransomware, stealthy spyware, and multi-stage trojans. These advanced threats are often designed explicitly to evade traditional security solutions, posing a significant and persistent risk to individuals, corporations, and national infrastructure. Traditionally, the first line of defense has been signature-based antivirus (AV) software. This approach, while effective against known threats, is fundamentally reactive and struggles against novel "zero-day" attacks. Malware authors now routinely employ techniques such as polymorphism and metamorphism to alter their code, generating countless new variants that bypass signature-based detection entirely. This inadequacy necessitates a paradigm shift towards more intelligent, proactive, and multi-layered security solutions. A robust defense must integrate multiple layers of analysis, including static analysis (inspecting a file's structure pre-execution) and dynamic analysis (observing a program's runtime behavior). Furthermore, the integration of machine learning (ML) has emerged as a powerful new frontier, enabling systems to learn from vast datasets and detect previously unseen threats with a high degree of accuracy. This thesis is motivated by the critical need for an advanced, hybrid defense system that combines these diverse methodologies to provide comprehensive, real-time protection.

1.3 Problem Description

The core problem addressed by this research is the inadequacy of security solutions that operate in isolated silos. Many endpoint protection platforms excel in one domain but fail to synergistically integrate these insights into a cohesive defense. A static scanner may identify a malicious file, but if it cannot act before execution, the damage is done. A behavioral monitor may flag a suspicious process, but without network context, it may miss command-and-control (C2) communications. These operational gaps create exploitable vulnerabilities, as an adversary needs to find only one such gap to compromise a system. The central challenge is to architect a framework that not only possesses diverse detection capabilities but also weaves them together into an integrated system where each component informs the others in real-time. This system must close the critical time window between detection and response by automating the neutralization of threats without requiring human intervention.

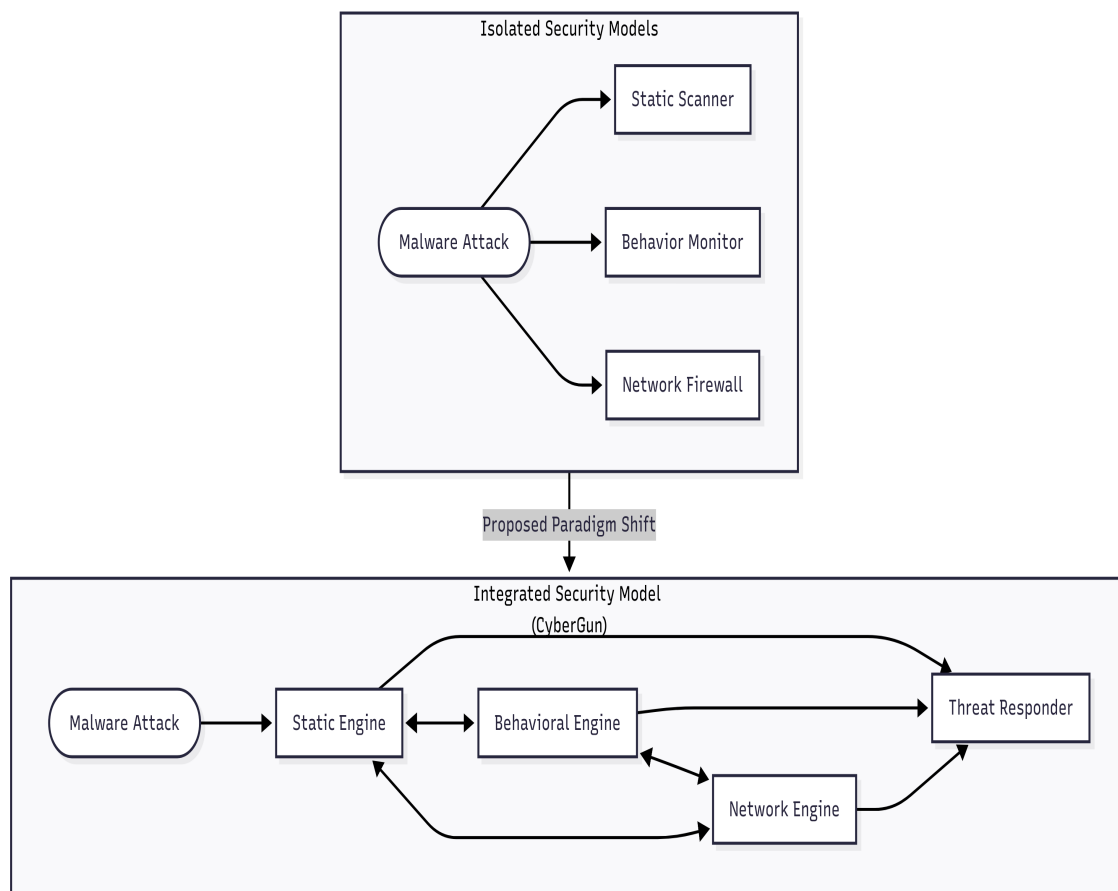


Figure 1.1: Isolated security models vs. integrated CyberGun architecture.

1.4 Proposed Solution: CyberGun

In response to the identified problem, this thesis proposes **CyberGun**, a modular, multi-layered framework for real-time malware detection and automated threat response. CyberGun is architected as a cohesive ecosystem of specialized engines designed to provide defense-in-depth. Its architecture is founded on three primary pillars of analysis:

1. **Static Engine:** A performance-optimized module that performs pre-execution analysis of files using a layered strategy. It combines high-speed hash matching [2], YARA rule evaluation [14], and machine learning classification models (e.g., LightGBM, Scikit-learn) [9]. This hybrid approach ensures rapid identification of known malware signatures while enabling the detection of novel or obfuscated threats that evade traditional signature methods.
2. **Dynamic Engine:** A real-time monitoring module that continuously observes system processes and resource usage. At its core is an ML-powered *behavioral analysis funnel* that applies lightweight system-state monitoring to trigger selective deep API call tracing using Frida [7]. This selective tracing strategy minimizes performance overhead while uncovering stealthy threats such as fileless malware, rootkits, and living-off-the-land (LotL) attacks.
3. **Network Engine:** A dedicated component that captures and analyzes network traffic in real time using packet inspection frameworks such as Scapy [4]. It inspects flows for connections to known malicious IP addresses [13], command-and-control patterns, or data exfiltration attempts. By integrating blacklist lookups with anomaly-based detection, this engine provides visibility into external communication channels often exploited by advanced persistent threats (APTs).

These engines are interconnected through a decoupled communication backbone **SignalBus** that allows them to share information and trigger actions seamlessly. When a high-confidence threat is detected, the **Threat Responder** is activated to immediately and automatically neutralize the threat.

1.5 Aims and Objectives

The primary aim of this study is to design, develop, and evaluate **CyberGun**, a hybrid cybersecurity framework capable of intelligent, real-time threat detection and automated response. The motivation behind this aim stems from the growing sophistication of malware, which increasingly bypasses traditional, isolated security solutions. To address this challenge, the framework must not only achieve high detection accuracy but also ensure efficiency, scalability, and adaptability in diverse environments.

The specific objectives of the study are as follows:

1. **To design a modular and resource-efficient architecture** that integrates multiple detection layers—static, dynamic, and network analysis—without compromising overall system performance. The architecture should employ techniques such as on-demand resource loading and parallel processing to ensure that security tasks run continuously in the background while maintaining responsiveness on consumer-grade hardware.
2. **To engineer a sophisticated, ML-powered behavioral analysis funnel** that begins with lightweight, system-state anomaly detection and progressively narrows down to targeted deep API tracing of suspicious processes. This hierarchical funnel approach ensures efficient resource usage while enabling the identification of advanced threats, including polymorphic and metamorphic malware, fileless attacks, and stealthy rootkits.
3. **To rigorously evaluate the framework’s performance and detection accuracy** using both benchmark datasets (e.g., EMBER [3], CIC-MalMem-2022 [5], MalAPI-2019 [6]) and curated real-world malware samples. The evaluation will focus on detection rate, false positive rate, resource utilization, and responsiveness, thereby validating the framework’s practicality in real-world endpoint security scenarios.
4. **To implement an automated, closed-loop threat response mechanism** that neutralizes detected threats in real-time by terminating malicious processes, quarantining files, and logging events for auditability. This ensures that detection

is immediately followed by mitigation, minimizing the window of exposure and reducing reliance on human intervention.

5. **To provide a user-centric interface and management environment** that enables both technical and non-technical users to monitor system health, view threat intelligence, and configure scanning policies. The goal is to balance automation with transparency, ensuring users remain informed while minimizing manual workload.

1.6 Key Contributions

Building upon the research objectives and the challenges outlined in the previous sections, this thesis makes contributions that advance both practical implementation and theoretical understanding in the domain of applied cybersecurity. The key contributions are as follows:

1. **A Novel Integrated Security Framework:** The central contribution of this work is the design and implementation of **CyberGun**, a modular, multi-layered framework that unifies static, dynamic, and network-based malware detection within a single operational environment. Unlike existing endpoint protection solutions that function in isolation, CyberGun demonstrates how diverse detection techniques can be orchestrated together through a centralized signal bus, enabling real-time information sharing and automated, closed-loop response.
2. **A Sophisticated, ML-Powered Behavioral Analysis Funnel:** This research introduces a new methodology for behavioral threat detection, structured as a hierarchical funnel. Broad system-state anomaly detection serves as a low-cost filter that selectively triggers deep API tracing of only the most suspicious processes. By combining lightweight classifiers with high-resolution tracing and ML-based sequence classification, this approach achieves both efficiency and precision, reducing false negatives while avoiding the performance costs of continuous system-wide instrumentation.

3. **A Resource-Efficient, On-Demand Architecture:** A key engineering contribution is the adoption of a “just-in-time” resource management strategy, particularly for static analysis components such as hash databases, YARA rules, and ML models. These resources are loaded only when required and released immediately afterward, ensuring a minimal memory footprint. This design makes CyberGun suitable for continuous real-time monitoring on standard consumer-grade hardware without compromising usability.
4. **Empirical Validation Using Benchmark Datasets and Real-World Samples:** The framework is rigorously evaluated on multiple benchmark datasets (EMBER [3], CIC-MalMem-2022 [5], MalAPI-2019 [6]) as well as curated real-world malware samples. Results demonstrate high detection accuracy with low false positive rates, providing empirical evidence of the effectiveness and practicality of the integrated approach.
5. **Bridging Research and Practice:** Beyond the implementation itself, this thesis contributes to the broader cybersecurity community by providing a proof-of-concept design that demonstrates how machine learning, modular architecture, and automated response can be integrated into endpoint protection. The framework serves as a reference model for future academic research and industrial development in adaptive, intelligent malware defense.

1.7 Project Significance

CyberGun addresses a critical gap in the cybersecurity domain by unifying static, dynamic, and network analysis into a single operational framework capable of real-time detection and response. Unlike conventional point solutions that operate in isolation, CyberGun’s integrated and modular architecture enables cross-context intelligence sharing between detection engines, significantly reducing the likelihood of missed threats. This holistic approach ensures that insights from one layer (e.g., file-level analysis) can directly inform and reinforce another (e.g., behavioral or network monitoring), creating a stronger, adaptive defense ecosystem.

The significance of this project can be understood in terms of its technical, practical, and societal impact:

1. **Enhancing early-stage threat detection:** By combining pre-execution static scanning with real-time behavioral and network monitoring, CyberGun can detect threats at multiple stages of their lifecycle, minimizing the window of exposure and reducing the chances of advanced malware bypassing defenses.
2. **Efficient resource utilization:** The framework employs on-demand resource loading and a behavioral funnel approach, ensuring that heavy operations such as deep API tracing are performed only when necessary. This makes the solution viable for both enterprise-grade systems and consumer endpoints with limited resources.
3. **Automated, closed-loop threat mitigation:** Unlike systems that simply generate alerts, CyberGun integrates automated response mechanisms such as process termination, file quarantine, and threat logging. This immediate action minimizes reliance on human intervention, reduces response latency, and prevents damage before it escalates.
4. **Adaptability to evolving threats:** The integration of machine learning models trained on benchmark datasets allows the system to generalize beyond known signatures. Furthermore, models can be retrained with fresh data, enabling the framework to evolve alongside new malware families and attack vectors.
5. **Practical applicability across domains:** CyberGun is designed for deployment in multiple contexts, including enterprise IT infrastructures, government and defense sectors, financial institutions, healthcare systems, and consumer devices. Its modular design and scalability make it adaptable to environments with diverse security needs.
6. **Contribution to research and practice:** Beyond its practical utility, this work provides a proof-of-concept implementation of a modular, ML-driven, multi-layered defense framework. It can serve as a foundation for future academic

research in intelligent endpoint protection and as a model for industry practitioners developing next-generation security tools.

In summary, CyberGun is significant not only as a technical innovation but also as a practical, resource-efficient, and socially impactful solution. It demonstrates that modern cybersecurity must move beyond isolated defenses toward integrated, intelligent systems capable of adapting to a constantly evolving threat landscape.

1.8 Project Scope

The scope of CyberGun is defined by its focus on the **detection, analysis, and automated mitigation** of malware threats in real time. The framework is designed to provide comprehensive endpoint security across three primary domains:

1. **Static Analysis:** Pre-execution inspection of files through a multi-layered approach. This includes hash-based matching for known malware, YARA rule scanning for family-level pattern recognition, and machine learning classification for previously unseen samples. The goal is to detect threats before they execute and to minimize the risk of compromise at the earliest stage.
2. **Dynamic Analysis:** Real-time behavioral monitoring of running processes and system interactions. A behavioral analysis funnel progressively narrows down from broad system-state anomaly detection to targeted deep API tracing of suspicious processes. This ensures that advanced threats such as polymorphic, metamorphic, and fileless malware can be identified even if they bypass static defenses.
3. **Network Analysis:** Continuous inspection of inbound and outbound network activity. This component focuses on detecting connections to malicious IP addresses, command-and-control (C2) servers, and abnormal traffic patterns. By correlating process-level behavior with network activity, the framework enhances its ability to detect stealthy attacks such as data exfiltration and botnet communications.

CyberGun is designed with broad applicability in mind, making it suitable for deployment in mission-critical environments where cybersecurity is of paramount importance, including but not limited to:

- **Financial institutions:** To prevent banking trojans, ransomware, and data exfiltration attacks.
- **Healthcare organizations:** To safeguard sensitive patient records and ensure uninterrupted availability of critical systems.
- **Government agencies:** To protect against espionage campaigns, state-sponsored malware, and insider threats.
- **Enterprise IT infrastructures:** To provide scalable protection across large-scale corporate environments with diverse endpoints.
- **Critical National Infrastructure (CNI):** To enhance resilience of energy, transportation, and communication systems against advanced persistent threats (APTs).

Inclusions: The project encompasses the full pipeline of threat management: malware detection (static, dynamic, network), classification using machine learning, automated response mechanisms such as process termination and file quarantine, and real-time event logging with a graphical user interface for monitoring and control.

Exclusions: CyberGun does not cover forensic investigation, attack attribution, or digital evidence preservation. It is not intended to perform deep packet inspection for encrypted traffic nor does it provide kernel-level monitoring in the current implementation. The focus is strictly on **proactive prevention and mitigation** rather than post-incident forensics.

Overall, the project scope emphasizes creating a practical, modular, and resource-efficient defense system that is scalable across consumer, enterprise, and critical infrastructure contexts while remaining strictly focused on prevention and real-time response.

1.9 Thesis Organization

The remainder of this thesis is organized into successive chapters to provide a logical flow of discussion. Chapter 2 presents the feasibility study and analysis, examining existing malware detection methodologies, their inherent limitations, and the emerging role of machine learning in modern cybersecurity. Chapter 3 outlines the requirement specifications of the proposed CyberGun framework, including its design rationale,

component interactions, and the communication backbone. Chapter 4 focuses on the system design and architecture, describing the development of the static, dynamic, and network detection engines, as well as the Threat Responder and the graphical user interface. Chapter 5 discusses the implementation and testing phase, covering the experimental setup, datasets, performance metrics, and comparative evaluation of results. Chapter 6 addresses the deployment aspects of the framework, highlighting practical considerations and operational insights. Finally, Chapter 7 concludes the thesis by summarizing the key findings, emphasizing the contributions made, and outlining potential directions for future research.

Chapter 2

Feasibility and Analysis

CyberGun addresses the shortcomings of current malware detection systems through a modular, multi-layered design that merges rapid static analysis with intelligent, targeted behavioral monitoring. Its architecture emphasizes low resource consumption, immediate automated threat response, and independence from constant cloud connectivity. A detailed feasibility assessment confirms its practicality, technical soundness, cost-effectiveness, and positive societal impact.

2.1 Existing Systems And their Drawbacks

The domain of endpoint security is populated by a wide range of solutions, from traditional signature-based antivirus (AV) products to modern Endpoint Detection and Response (EDR) platforms. Traditional AV solutions, such as Avast, AVG, and ClamAV, primarily operate by matching file hashes and byte sequences against vast databases of known malware. Modern EDR solutions and academic frameworks, like CrowdStrike Falcon and Cuckoo Sandbox, move beyond static signatures to incorporate behavioral analysis and machine learning. Despite their advancements, existing systems exhibit several critical drawbacks that CyberGun aims to address:

- **Architectural Monoliths:** Many commercial solutions operate as proprietary “black boxes,” which conceals their internal decision-making processes and makes it difficult to understand or verify their detection logic. On the other hand, academic sandboxing systems, while often more transparent, tend to be resource-intensive and cumbersome, making them impractical for continuous, real-time monitoring

on typical user endpoints. As a result, deploying these solutions at scale without affecting system performance remains a significant challenge.

- **Resource Intensity:** Continuous deep behavioral monitoring can impose a substantial load on system resources, consuming significant CPU cycles and memory. This can degrade overall system performance, making such solutions impractical for many consumer-grade devices or other resource-constrained environments.
- **Lack of Integrated, Automated Response:** Many detection systems generate alerts without providing an immediate, automated mechanism to neutralize the threat. This creates a critical time gap during which malware can continue to operate, propagate, or exfiltrate data before human intervention occurs.
- **Over-reliance on Cloud Connectivity:** Numerous modern security solutions rely heavily on constant access to cloud-based threat intelligence for analysis and updates. In air-gapped networks or environments with intermittent connectivity, this dependence can severely limit the effectiveness of the protection mechanisms.

2.2 Proposed System and its features

In response to the limitations of existing systems, this project proposes **CyberGun**, a modular, multi-layered framework for real-time malware detection and automated threat response. The core design philosophy is to create a hybrid system that leverages the speed of static analysis for known threats while employing an intelligent, targeted approach for dynamic analysis of unknown threats. By operating directly on the endpoint, it provides a self-contained and resilient layer of defense.

- **Multi-Layered Static Analysis:** Integrates high-speed hash matching, YARA rule scanning, byte-level signature detection, and a machine learning classifier to achieve comprehensive, accurate static threat detection.
- **On-Demand Resource Management:** Implements a “just-in-time” resource loading strategy for all static scanners, minimizing memory usage during idle periods and reducing overall system overhead.

- **Intelligent Behavioral Analysis Funnel:** Leverages a lightweight system-state classifier to selectively trigger deep, targeted API call tracing only on processes exhibiting suspicious behavior, thereby avoiding the performance cost of full system-wide monitoring.
- **Decoupled and Modular Architecture:** Built around a central SignalBus for asynchronous communication, enabling easy maintenance, modular extension, and scalable deployment of additional detection components.
- **Automated “Closed-Loop” Threat Response:** Directly connects detection to a ThreatResponder module, allowing immediate automated actions such as process termination and file quarantine, effectively reducing the window of exposure to malware.

2.3 Feasibility Report of Proposed System

A feasibility analysis was conducted to evaluate the viability of the CyberGun project. The results indicate that the project is achievable with the available resources and provides a practical, effective solution to the challenges identified in endpoint security.

2.3.1 Technical Feasibility

The project is technically feasible. The framework is implemented in Python, leveraging a mature ecosystem of well-established open-source libraries, which significantly mitigates development risk.

- **Core Libraries:** The technical foundation relies on PyQt5 for the graphical user interface, psutil for system monitoring [11], Frida for dynamic API tracing [7], YARA for rule-based malware scanning [14], and LightGBM/Scikit-learn for machine learning components [12]. These libraries are widely used, stable, and actively maintained, ensuring reliability and long-term support.
- **ML Model Viability:** The intelligent detection components are trained and validated using benchmark datasets, including EMBER [3], CIC-MalMem-2022 [5],

and MalAPI-2019 [6]. This ensures that the models are based on proven, state-of-the-art research and provide accurate threat detection.

- **Hardware Requirements:** The application’s resource-efficient design allows it to operate effectively on standard consumer-grade hardware, minimizing CPU and memory overhead and avoiding significant performance degradation during operation.

2.3.2 Economic Feasibility

The project is economically feasible because it requires only minimal financial investment, relying primarily on openly available resources. Python, the chosen development language, along with widely used open-source libraries, eliminates the need for costly software licenses. Similarly, the use of public datasets for model training removes expenses related to data acquisition. Moreover, the framework does not depend on specialized or expensive hardware, meaning that its successful implementation depends mainly on dedication, time, and effort rather than significant monetary resources.

2.3.3 Social Feasibility

The project is socially and ethically sound, as it is designed to deliver clear benefits while adhering to responsible practices. Its primary objective is to enhance cybersecurity and protect users from malicious attacks, thereby contributing positively to society. Ethical considerations have been carefully addressed by relying on publicly available and anonymized datasets, ensuring that no sensitive information is compromised. Moreover, all monitoring is performed locally on the user’s machine without transmitting personal data externally, which safeguards privacy. Finally, the tool is developed solely for defensive purposes, reinforcing its alignment with ethical and socially responsible use.

Chapter 3

Requirement Specification

The CyberGun framework is designed to deliver robust endpoint protection through a combination of multi-layered scanning and automated threat response mechanisms. Its requirements encompass functional capabilities such as static and dynamic analysis, real-time behavioral monitoring, and immediate containment of malicious activity. Non-functional aspects—performance, reliability, and usability—define the quality standards the system must maintain under operational conditions. The specification also includes user interface requirements, a defined target user persona, and system interaction models (supported by a use case diagram). Additionally, the framework’s boundaries and environmental dependencies are clearly established to ensure smooth development, deployment, and long-term maintainability.

3.1 Functional Requirements

Functional requirements define the specific behaviors and capabilities of the CyberGun system. They describe what the system must do to fulfill its purpose as a comprehensive security tool.

3.1.1 Multi-Layered Static Analysis

To ensure robustness against diverse malware types, the system employs a multi-layered static analysis approach. Each layer complements the others by combining lightweight signature-based techniques with advanced machine learning models. This layered strategy enhances detection accuracy while minimizing false positives and ensuring efficient

performance.

- The system shall be able to scan individual files and entire directories on demand.
- The system shall implement a hash-based scanner to detect known malicious files by comparing their SHA-256 [10] hashes against a threat database.
- The system shall integrate the YARA engine to scan files against a ruleset for identifying malware families based on textual and binary patterns.
- The system shall utilize a pre-trained machine learning model to classify Windows Portable Executable (PE) files as either benign or malicious.
- The system shall include scanners for detecting malicious byte signatures and suspicious strings within files.

3.1.2 Real-Time System Monitoring

Real-time monitoring is a critical component of CyberGun's proactive defense model. Instead of relying solely on scheduled scans or user-initiated checks, the framework observes system activity, device interactions, and process behavior continuously. This ensures that threats are detected and mitigated at the earliest possible stage, reducing the risk of compromise. When combined with the behavioral analysis funnel, real-time monitoring provides both broad situational awareness and targeted deep inspection.

- **Filesystem Monitoring:** The system shall continuously monitor specified filesystem directories (e.g., C:/) for file creation, deletion, or modification events. Upon detecting such activity, the system shall immediately trigger an on-demand static scan of the affected file(s) to prevent execution of newly introduced malware.
- **Removable Media Monitoring:** The system shall detect the connection of new USB storage devices and automatically initiate a full scan of the device contents. This prevents malicious executables from propagating through infected portable media, which remains a common vector for malware distribution.
- **Process Metadata Monitoring:** The system shall continuously monitor system-wide process metadata, including process identifiers, parent-child relationships,

CPU and memory usage, thread counts, and execution duration. These features shall be logged and fed into the behavioral analysis engine to identify anomalies that may indicate stealthy or fileless malware.

- **Behavioral Analysis Funnel:** The system shall implement a multi-stage behavioral analysis funnel. Low-cost monitoring of system states (e.g., abnormal CPU spikes or suspicious process spawning) shall act as a trigger for high-cost, deep API call tracing of selected processes. This hierarchical funnel approach balances resource efficiency with the ability to detect advanced threats such as polymorphic malware, rootkits, and living-off-the-land (LotL) attacks.
- **Automated Response Integration:** The system shall ensure that real-time monitoring is directly coupled with the automated response mechanism. When anomalous behavior is confirmed, the framework shall immediately terminate the offending process, quarantine associated files, and log the event for future analysis.

3.1.3 Network Threat Detection

Network monitoring is essential for identifying malicious communications that may bypass local file or process-based defenses. By capturing and analyzing live traffic, the system can detect suspicious connections in real time and prevent data exfiltration or contact with command-and-control servers. This layer of protection ensures that even stealthy threats attempting to communicate externally can be identified and blocked.

- The system shall be capable of capturing live network traffic from a specified network interface.
- The system shall analyze captured packets and check the destination IP addresses against a blacklist of known malicious servers.
- The system shall generate an alert when a connection to a blacklisted IP address is detected, and (where possible) correlate the network event with the originating process.

3.1.4 Automated Threat Response & Management

Detection alone is insufficient without a timely and effective response. To minimize the impact of threats, the system incorporates automated mechanisms that immediately contain and neutralize malicious activity. This ensures not only proactive defense but also provides auditability and adaptability through continuous updates of intelligence sources.

- Upon detection of a high-confidence threat, the system shall automatically terminate the associated malicious process (respecting system stability and safe-termination procedures).
- Following process termination, the system shall automatically move the malicious file to a secure, isolated quarantine directory and preserve a copy for analysis if permitted.
- The system shall provide a mechanism for updating its threat intelligence databases (hash lists [2] [1], YARA rules [15], IP blacklists [13]) from remote sources, with integrity checks on downloaded artifacts.
- The system shall log all detection events and system actions to persistent log files for auditing and review, and provide facilities to export logs in common formats (CSV/JSON).

3.2 Non-Functional Requirements

The non-functional requirements of the system describe how the system should behave, rather than what functions it must perform. These requirements define the quality attributes and operational characteristics that ensure the CyberGun system remains practical, efficient, and dependable in real-world use.

3.2.1 Performance

Performance requirements ensure that the system runs efficiently without significantly degrading the overall user experience. They emphasize speed, responsiveness, and

lightweight resource utilization.

- **Resource Efficiency:** The system should aim to maintain a low CPU (approximately 5%) and memory (approximately 200 MB) footprint during idle monitoring; target values should be validated empirically during testing and adjusted as needed.
- **Responsiveness:** The Graphical User Interface (GUI) must remain responsive and not freeze, even during intensive background tasks such as full-directory scans.
- **Scan Speed:** The static scanning engine should be optimized for high throughput, capable of processing a large number of files in a reasonable timeframe; expected throughput and benchmarks should be reported in the evaluation chapter.

3.2.2 Reliability & Stability

Reliability ensures that the system consistently delivers its intended services, while stability guarantees uninterrupted monitoring even in the presence of unexpected conditions. This is essential for a security tool that may need to operate continuously in the background.

- The system shall be designed for continuous, long-term operation without crashing or requiring frequent restarts.
- The system must handle errors gracefully (e.g., file access errors, network timeouts) and log them without terminating the entire application.
- Key modules should implement retries and circuit-breaker patterns for external actions (e.g., updates, remote downloads).

3.2.3 Usability

Usability requirements ensure that CyberGun remains user-friendly and accessible to both technical and non-technical users. The focus is on clear presentation of information, intuitive navigation, and actionable feedback for detected threats.

- The GUI shall provide a clear, intuitive, and centralized dashboard for viewing threats, system status, and logs.
- Alerts and logs must be presented in a human-readable format, providing clear and actionable information.
- The system shall minimize unnecessary user interactions, automating threat responses while still allowing manual overrides and user confirmation for critical actions.

3.2.4 Maintainability & Scalability

Maintainability ensures that the system can be efficiently updated, debugged, and improved over time, while scalability guarantees that the architecture can grow to handle larger workloads without major redesign.

- The system's architecture must be modular, allowing individual components (e.g., a new scanner) to be added, updated, or removed with minimal impact on the rest of the framework.
- Configuration files, rules, and machine learning models shall be easily updatable without requiring a full reinstallation of the system.
- The system must scale to handle increased monitoring scope (e.g., larger directories, more network traffic) without significant performance degradation.

3.3 UI Requirements

The UI requirements define the key components and interactive elements of the Cyber-Gun graphical interface. They ensure that users can monitor system status, view alerts, and interact with threat management features efficiently.

3.3.1 Main Dashboard View

The dashboard is the primary control center, designed to provide a real-time overview of threats and system activity.

- Shall display a primary feed for high-priority static scan detections.
- Shall include a summary panel for real-time system health (CPU and memory usage).
- Shall feature a live event feed for displaying real-time activities like file system changes and USB detections.
- Shall feature a feed showing live updates about intelligence sources such as malwarebazaar [2] and abuse.ch [1].
- Shall provide tabs for prioritized feeds (Behavior, Network, Threats, System Status) and a Configuration tab.

3.3.2 Other Views

In addition to the main dashboard, the system shall provide specialized views to give users deeper insights into system behavior, threat activity, and historical records.

- **System Status Dashboard:** A dedicated view for detailed, real-time metrics on CPU, memory, disk I/O, network traffic, and a sortable, interactive list of all running processes.
- **Threat Log Viewer:** A view for displaying the persistent, historical log of all detected threats, allowing filtering and searching by type, severity, or timestamp.
- **Behavior and Network Feeds:** Dedicated views for displaying real-time alerts from the dynamic behavioral and network analyzers in a structured feed format.

3.3.3 Controls and Interactivity

The UI shall support interactive controls to empower users with manual actions and customization options when dealing with system monitoring and threat responses.

- The UI shall provide buttons to manually initiate a full-directory scan, stop an ongoing scan, and update all threat intelligence databases.

- Users shall be able to right-click a process in the System Status view to terminate it immediately (with confirmation for critical system processes).
- Users shall be able to export system and threat logs to a local file in common formats (e.g., CSV, JSON, PDF).
- The system shall support adjustable notification settings (e.g., popup alerts, sound, or silent logging) to accommodate different user preferences.

3.4 User Interactions with the System through Use Case Diagram

The use case diagram in Figure 3.1 illustrates how the primary actor, the *User/Administrator*, interacts with the CyberGun system. It highlights key functionalities such as initiating on-demand scans, updating threat intelligence, viewing threat logs and system status, and configuring application settings. The system is also shown performing automated actions like neutralizing detected threats. External elements such as the operating system, hardware devices, and remote update servers remain outside the system boundary but interact with CyberGun through these use cases.

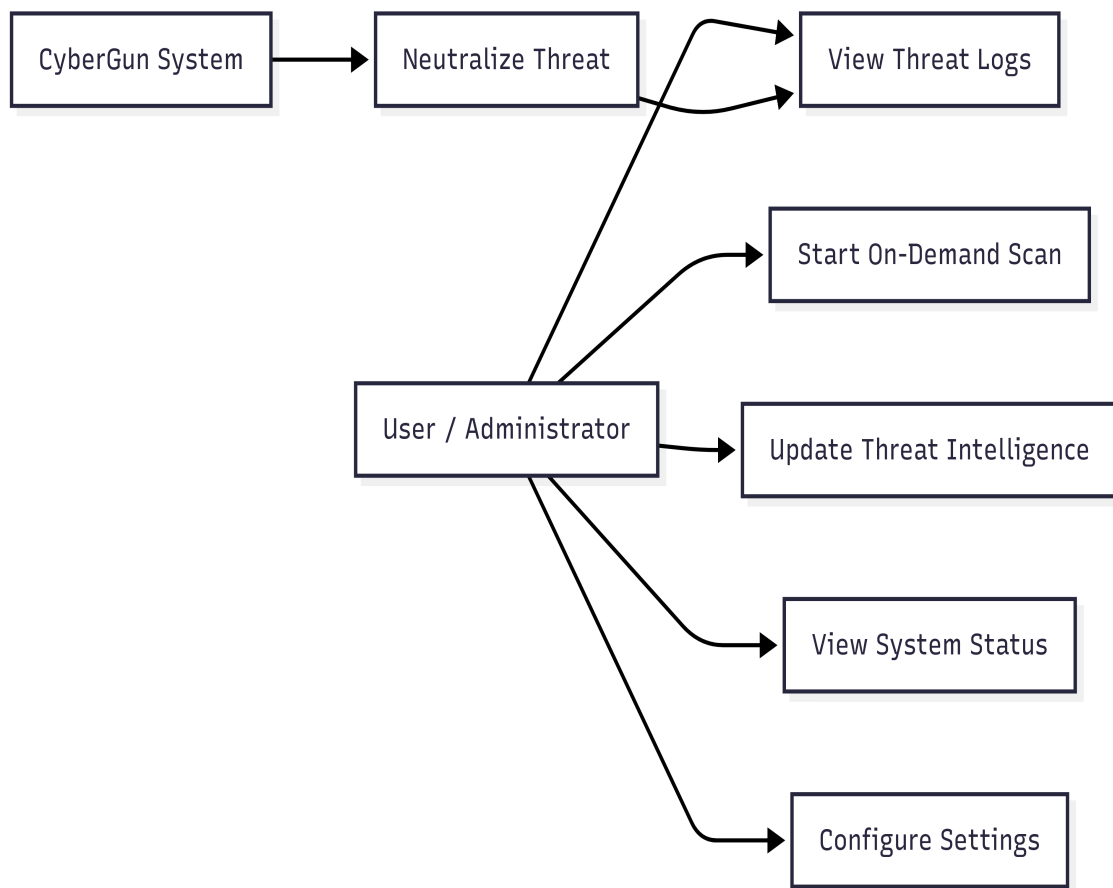


Figure 3.1: Use Case Diagram of the CyberGun System.

3.4.1 System Boundary

The system boundary defines the exact scope of the CyberGun application in relation to its operating environment. It distinguishes between components that are internal to CyberGun and those that belong to the host system or external world. This boundary is important for clarifying the responsibilities of CyberGun and identifying where external interactions occur. It ensures that both developers and users understand what the application controls and what it merely observes.

Within the System Boundary

- The CyberGun application executable and its Python source code.
- All core engines (StaticEngine, DynamicEngine, NetworkEngine).
- All scanner, monitor, and manager modules.

- The Graphical User Interface (GUI).
- Locally stored threat intelligence databases (hash lists, YARA rules, signatures).
- The quarantine directory and log files.

Outside the System Boundary (Environment)

- The User (Administrator).
- The Windows Operating System.
- The host machine's filesystem, running processes, and network stack.
- External hardware (e.g., USB drives).
- Remote servers and repositories that host threat intelligence updates.

3.4.2 Development Environment Requirements

The development environment requirements describe the resources needed by developers to build, maintain, and extend the CyberGun project. They cover both the hardware specifications required for efficient performance and the software tools necessary for coding and debugging. Ensuring a consistent development environment helps avoid compatibility issues across different setups. These requirements also provide a baseline for collaborative work and version control.

Hardware

- **Processor:** x86-64 multi-core processor (e.g., Intel Core i5/i7, AMD Ryzen 5/7).
- **RAM:** Minimum 8 GB DDR4, recommended 16 GB DDR4 for multitasking and handling large datasets.
- **Storage:** Minimum 5 GB of free space for source code, dependencies, and datasets.

Technology Stack (Software)

The technology stack outlines the software environment and tools required for the development and execution of the CyberGun application. It includes the operating system, programming language, libraries, and supporting utilities needed to ensure functionality and maintainability. Using a well-defined stack helps standardize the development process, simplifies debugging, and provides compatibility across different setups. This ensures developers can work in a consistent and reliable environment.

- **Operating System:** Windows 10 or Windows 11.
- **Programming Language:** Python 3.10+.
- **Core Libraries:** PyQt5, psutil, yara-python, lightgbm [9], scapy, frida, watchdog, requests, GitPython.
- **External Tools:** Git, Wireshark (for TShark).
- **Development Tools:** Visual Studio Code or PyCharm (recommended), pip/venv for dependency management.
- **Version Control:** Git (with GitHub/GitLab integration).

3.4.3 Deployment Requirements

Deployment requirements outline the conditions under which the CyberGun application can be successfully installed and executed on a target machine. These requirements ensure that the software has sufficient resources, proper privileges, and the necessary supporting tools for stable operation. Clearly defining deployment prerequisites minimizes compatibility issues and simplifies setup for administrators. This section specifies the mandatory operating system, hardware capabilities, dependencies, and configuration steps needed for smooth deployment.

- **Operating System:** Windows 10 (64-bit) or Windows 11.
- **System Privileges:** Administrator rights are required for system-wide process monitoring, API tracing with Frida, and network packet capture.

- **Hardware Requirements:**

- Minimum: Dual-core CPU, 4 GB RAM, 500 MB free disk space.
- Recommended: Quad-core CPU, 8 GB RAM, 1 GB free disk space for logs and updates.

- **Dependencies:**

- A compatible Python 3 runtime environment (version 3.10 or later).
- Wireshark must be installed, with TShark's location added to the system's PATH environment variable or specified in the settings file.
- Frida installed and accessible in the Python environment for API tracing.

- **Installation Method:** The system shall be provided as an installer package or service deployment script that automatically configures dependencies and registers the application as a Windows service.

- **Post-Deployment Configuration:** After installation, users must configure exclusion lists, update threat intelligence databases, and (optionally) schedule periodic scans via the settings panel.

Chapter 4

System Design and Architecture

The design of the CyberGun system translates functional requirements into a modular and scalable security solution. Its architecture separates the graphical interface from backend detection engines to ensure responsiveness during continuous monitoring. Data handling leverages structured files, compiled rule sets, and serialized machine learning models, loaded on demand to optimize performance. Distinct engines for static, dynamic, and network analysis operate under a centralized signal bus for coordination. This structure enables real-time threat detection, efficient response mechanisms, and intuitive control for the operator.

4.1 Architecture Design

The CyberGun framework employs a decoupled, multi-threaded architecture that ensures a responsive user interface while continuously executing computationally intensive security tasks in the background. The design is centered on a strict separation between the Graphical User Interface (GUI) and the backend analysis engines, with all interactions coordinated through a centralized messaging layer. To meet the functional and non-functional requirements, several key architectural choices were made:

1. **GUI and Backend Separation:** The GUI operates on the main thread, while all scanning and monitoring processes run in dedicated worker threads. This approach preserves interface responsiveness during heavy processing, which is critical for real-time monitoring.

2. **Centralized Signal-Slot Communication:** A singleton *SignalBus*, based on PyQt’s signal-slot mechanism, enables asynchronous, thread-safe communication between components. This decoupled messaging pattern improves modularity and maintainability.
3. **Engine-Based Orchestration:** The backend is divided into specialized orchestrator classes—*StaticEngine*, *DynamicEngine*, and *NetworkEngine*—to clearly separate file scanning, behavioral monitoring, and network analysis responsibilities.
4. **On-Demand Resource Loading:** Large databases (hashes, YARA rules, ML models) are loaded into memory only when required and released afterward, minimizing the system’s memory footprint during idle periods.

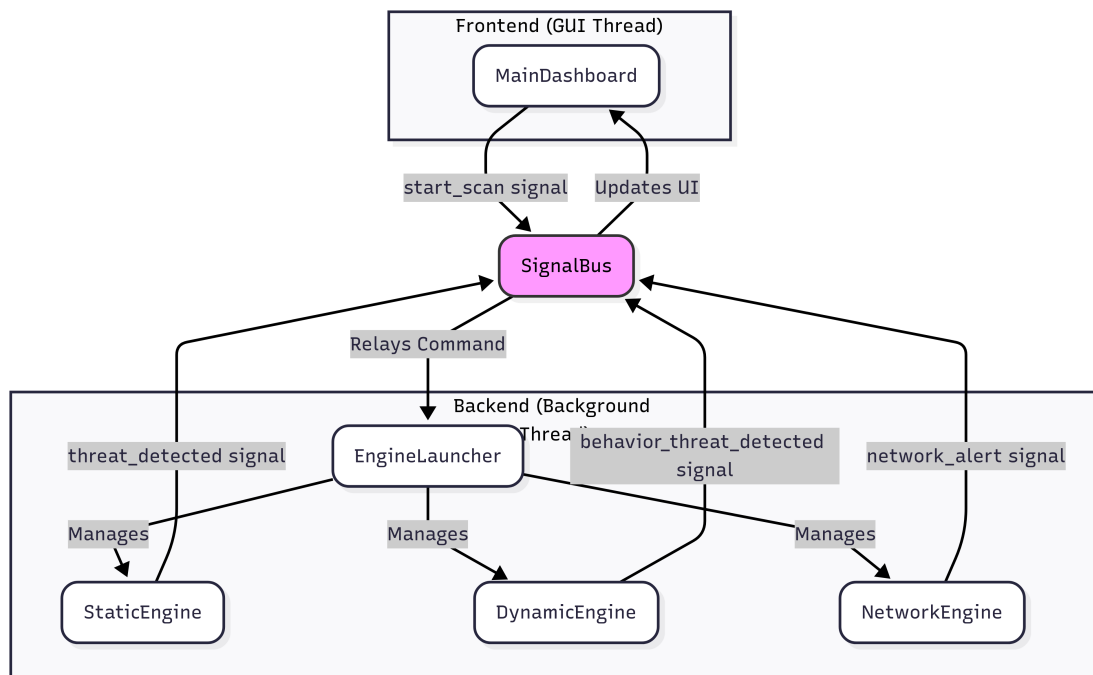


Figure 4.1: Architecture of the proposed system

Figure 4.1 illustrates the high-level, event-driven architecture of the CyberGun system, emphasizing the clear separation between the presentation layer (Frontend) and the processing layer (Backend). The **Frontend**, which contains the MainDashboard, operates exclusively on the main GUI thread to ensure a fluid and responsive user experience. All user interactions, such as initiating a scan, are translated into signals. The core of this decoupled architecture is the **SignalBus**, a centralized communication channel that

facilitates asynchronous messaging between the frontend and backend. This design prevents the GUI from freezing during intensive operations. The **Backend** operates in one or more background threads and contains the core logic. An **EngineLauncher** acts as the primary controller, managing specialized engines for static, dynamic, and network analysis. When these engines detect threats, they emit signals (e.g., `threat_detected` signal) which are broadcast through the SignalBus. The MainDashboard subscribes to these signals to update the UI with real-time status changes and alerts, while the EngineLauncher listens for commands from the GUI to manage the backend operations. This model enhances modularity, scalability, and maintainability by ensuring that components interact through well-defined signals rather than direct calls.

4.2 Data Persistence and Data Access Design

The “data” in CyberGun primarily consists of threat intelligence (signatures, rules, blacklists) and generated logs. The design for accessing this data prioritizes speed for detection and simplicity for management.

4.2.1 Data Persistence Design Details

- **Threat Intelligence:** All threat intelligence is stored as flat files in a structured datasets directory.
 - **Hashes:** Stored in plain text files (`.txt`) for fast loading into a Python set for $\mathcal{O}(1)$ average time complexity lookups.
 - **YARA Rules:** Stored as standard `.yar` files, which are compiled into a highly efficient binary format at runtime by the `yara-python` library.
 - **IP Blacklists:** Stored as a simple newline-delimited text file for easy parsing and loading.
- **Machine Learning Models:** Trained models are serialized to disk using the `joblib` [8] library (`.joblib`) for static analysis and `.lgbm` for the LightGBM models. This allows pre-trained models to be loaded quickly at runtime without the need for retraining.

- **Logs:** System and threat logs are persisted as plain text files (`.log`) in a dedicated logs directory. A rotating file handler is used to manage log size automatically.
- **Configuration:** System settings are managed within the `settings.py` file, providing a centralized location for configuring paths, feature toggles, and thresholds.

4.2.2 Data Access Design Details

- **Threat Intelligence Access:** Data access is handled by the respective scanner modules. For example, `HashDetector.py` is responsible for opening and parsing the hash files, while `YaraDetector.py` is responsible for finding and compiling the YARA rule files. This encapsulates the data access logic within the component that uses it.
- **On-Demand Loading:** The `load_resources()` and `unload_resources()` methods within each static scanner constitute the core of the data access strategy. This ensures that large datasets are only held in memory for the minimum time necessary.
- **Logging Access:** The `logger.py` utility provides a thread-safe interface for writing to the log files, preventing corruption from concurrent write operations from different parts of the application.

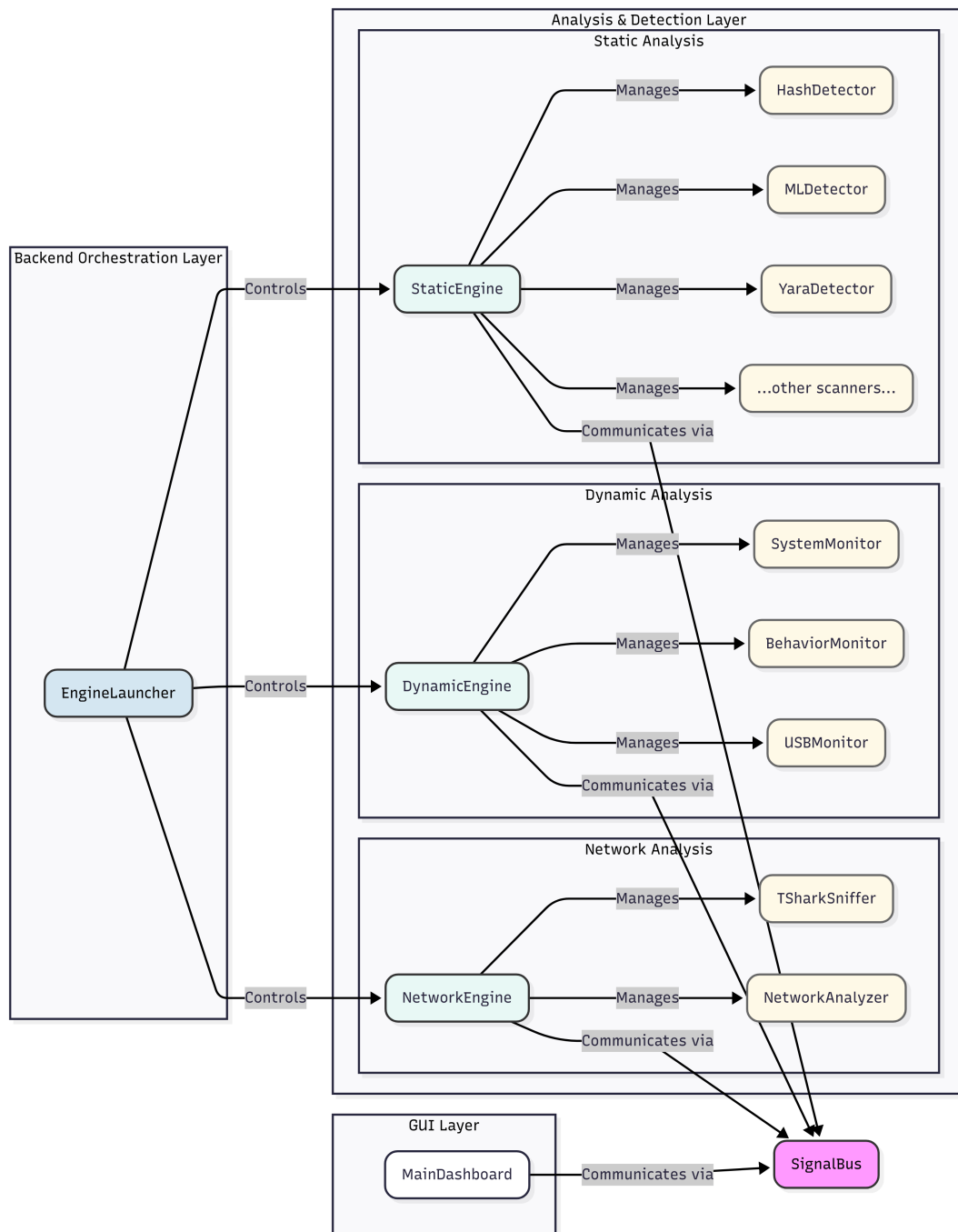


Figure 4.2: Component Diagram of CyberGun showing core modules and their interactions.

Figure 4.2 presents a detailed component-based view of the system, illustrating the principles of hierarchical control and separation of concerns. The architecture is organized into distinct layers. The **Backend Orchestration Layer** contains the **EngineLauncher**, which serves as the central controller responsible for initiating and managing the lifecycle of the various analysis engines. The **Analysis & Detection Layer**

is subdivided into three specialized domains: Static, Dynamic, and Network Analysis. Each domain is governed by a corresponding engine (`StaticEngine`, `DynamicEngine`, `NetworkEngine`) that acts as a manager for a suite of specialized detector and monitor components. For example, the `StaticEngine` manages scanners like `HashDetector` and `MLDetector`, while the `DynamicEngine` oversees monitors for system, behavior, and USB events. This hierarchical structure promotes modularity, as new scanners or monitors can be added with minimal impact on the rest of the system. Communication flows from the high-level orchestrators to the specialized components. All detection components from the various engines communicate their findings asynchronously to the **GUI Layer** through the central `SignalBus`, ensuring a decoupled and scalable information flow.

4.3 Component Design

The system is structured into modular, self-contained components, each with a clearly defined responsibility. This modular design allows for easier maintenance, scalability, and future enhancements. Each component communicates with others through a well-defined signaling mechanism (`SignalBus`), ensuring decoupling and robust event-driven operation.

4.3.1 System Workflow

The system follows a continuous, event-driven workflow orchestrated by the `EngineLauncher`. The workflow can be broken down as follows:

1. **Initialization:** Upon startup, `main.py` initializes the graphical user interface (GUI) and moves the `EngineLauncher` to a background thread to prevent blocking the main application. This ensures that the user interface remains responsive while the system performs background monitoring and scanning.
2. **Backend Activation:** The `EngineLauncher` starts all core backend services:
 - `DynamicEngine`, which launches both the `SystemMonitor` and `BehaviorMonitor` for continuous behavioral analysis.

- **NetworkEngine**, responsible for monitoring network traffic to detect anomalies or malicious activity.
- **FileSystemMonitor**, which observes the file system for new, modified, or deleted files.

3. **Initial Setup:** The system performs one-time preparatory tasks:

- Updates threat intelligence databases via **ThreatDownloader** to ensure the latest malware signatures are available.
- Conducts a full, layered static scan of the default directory to establish a clean baseline and detect any pre-existing threats.

4. **Continuous Monitoring:** After initialization, the system enters a passive monitoring state while continuously analyzing events:

- **FileSystemMonitor** and **USBMonitor** watch for changes in files and external devices, triggering the **StaticEngine** as needed.
- **SystemMonitor** and **BehaviorMonitor** continuously analyze process behavior and system activity for signs of malicious patterns.
- **NetworkEngine** inspects incoming and outgoing network traffic in real-time for suspicious connections or data exfiltration attempts.

5. **Threat Detection & Response:** When any monitoring component detects a potential threat:

- A signal (e.g., `threat_detected`) is emitted to the **SignalBus**, ensuring all subscribed components are notified.
- **ThreatResponder** evaluates the severity and, if necessary, initiates remediation actions such as terminating malicious processes and quarantining infected files using the **QuarantineManager**.
- The **MainDashboard** updates the user interface to reflect new threat information, providing immediate visibility to the user.

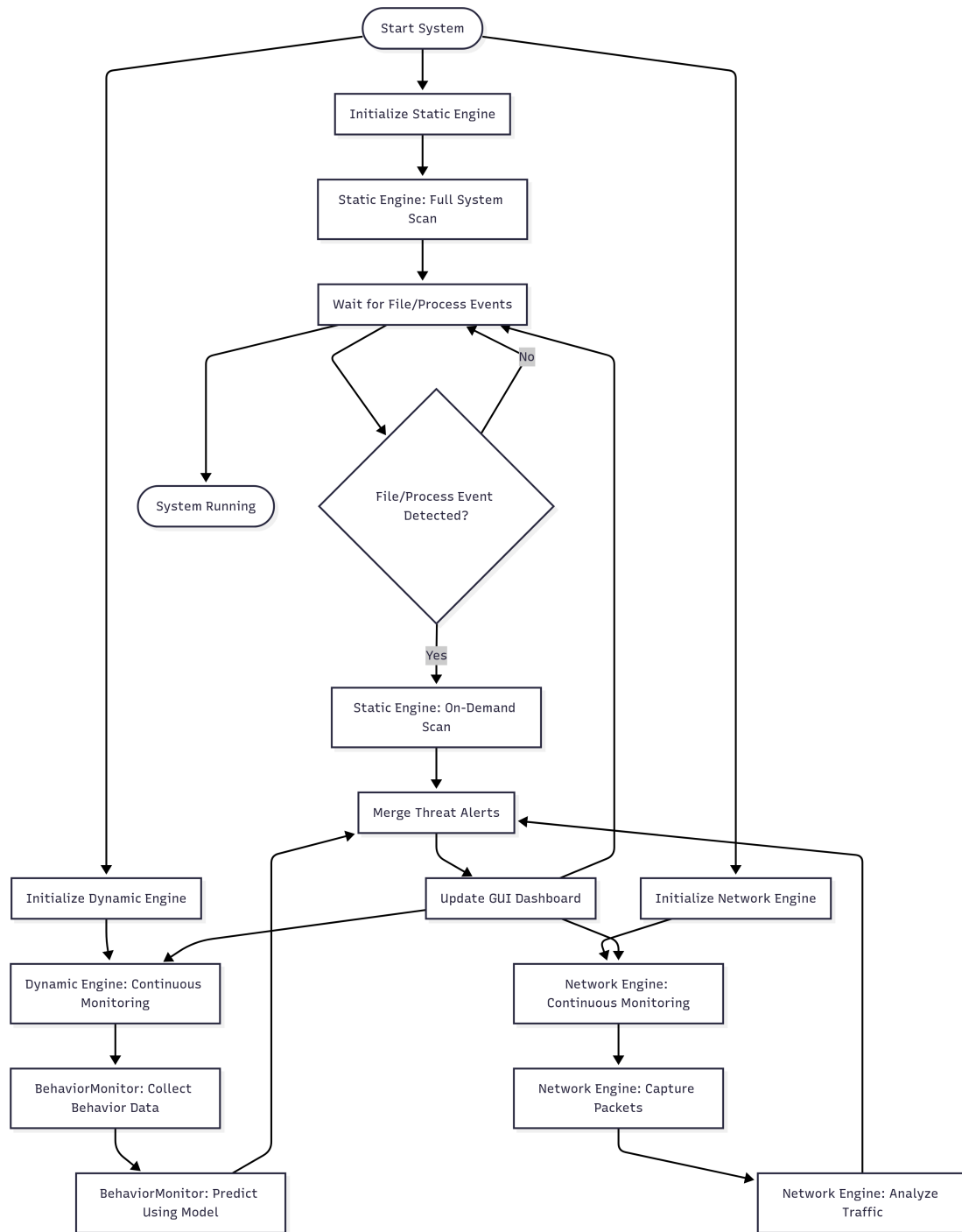


Figure 4.3: An Activity Diagram illustrating system workflow.

The activity diagram in Figure 4.3 illustrates the **layered defense strategy** employed by the static analysis engine. This workflow follows a *fail-fast* principle, where computationally inexpensive checks are performed first to quickly identify known threats.

The process begins by **collecting all files** from the target directory. **Layer 1** performs a **hash-based scan** (HashDetector), which checks file hashes against a database of

known malware signatures. This operation is extremely fast, with $O(1)$ average time complexity. If a threat is detected, the file is immediately flagged, and scanning for that file terminates, skipping all subsequent layers.

Files that pass Layer 1 proceed to **Layer 2**, the **Yara-based scan** (YaraDetector), which uses pattern-matching rules to detect malware families. Files not flagged here continue to **Layer 3**, the **signature-based scan** (SignDetector), which compares file signatures against a curated set of threat signatures.

Layer 4, the **string-based scan** (StringDetector), analyzes textual patterns within files that may indicate malicious intent. Finally, **Layer 5** performs a **machine learning-based analysis** (MLDetector), using a pre-trained model to predict the likelihood of malicious behavior based on extracted file features.

This **layered approach** optimizes resource usage by ensuring that deeper, more computationally expensive analyses are only applied to files that have passed the initial, faster layers, improving overall scanning efficiency while maintaining high detection coverage.

4.3.2 System Workflow

The data flow within CyberGun is designed to transform raw system events and file data into actionable security intelligence.

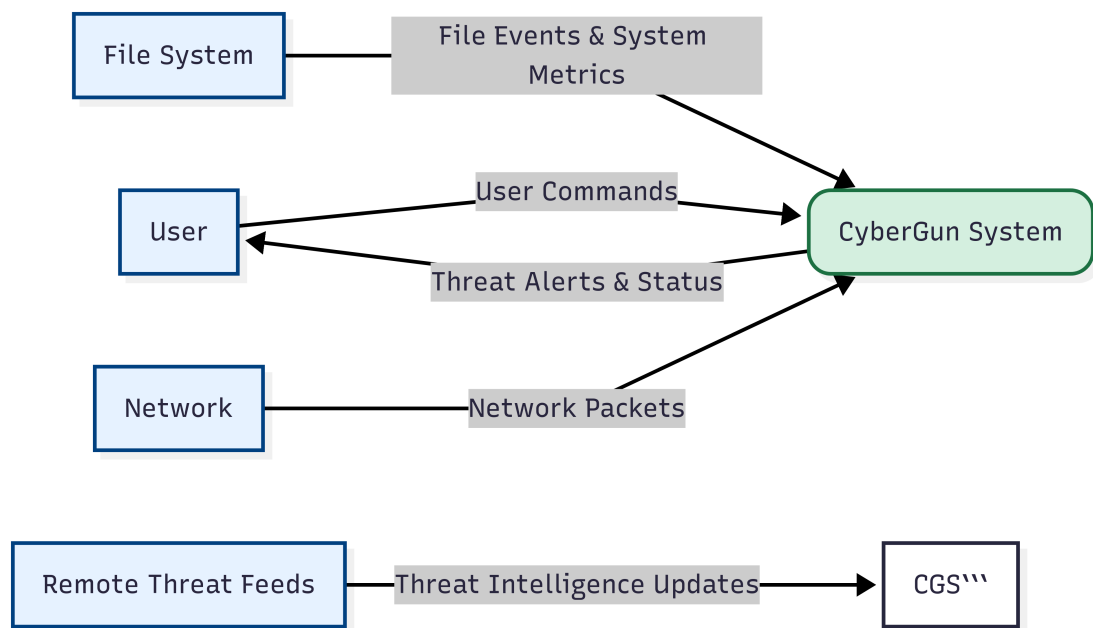


Figure 4.4: Data Flow Diagram (DFD) - Level 0.

Figure 4.4 shows a Level 0 Data Flow Diagram, also known as a System Context Diagram. This diagram provides a high-level overview of the CyberGun system, treating the entire application as a single process to illustrate its interactions with external entities. It defines the boundaries of the system and the primary inputs and outputs. The external entities interacting with the **CyberGun System** are: the **File System**, which provides file events and system metrics; the **User**, who issues commands and receives threat alerts and status updates; the **Network**, which is the source of network packets for analysis; and **Remote Threat Feeds**, which supply threat intelligence updates to keep the system's detection capabilities current. This diagram is crucial for understanding the system's scope and its relationship with its operational environment.

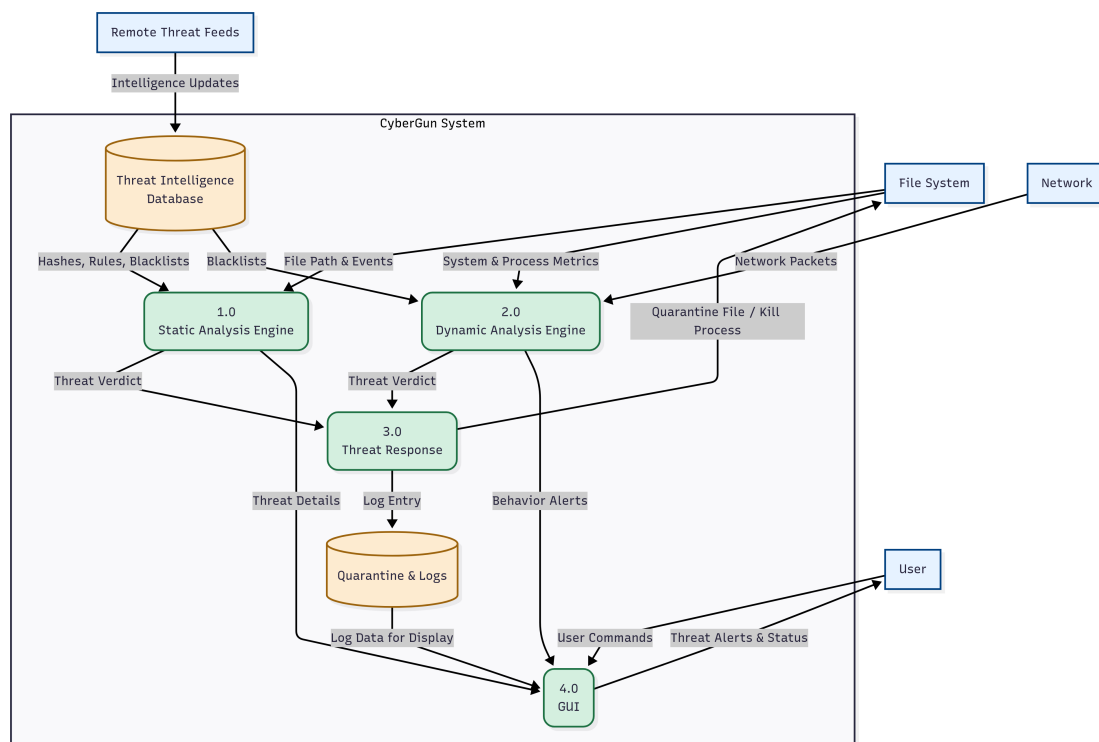


Figure 4.5: Data Flow Diagram (DFD) - Level 1.

Figure 4.5 presents a Level 1 Data Flow Diagram, which decomposes the single CyberGun process from the Level 0 diagram into its major functional components and illustrates the flow of data between them. The process begins with **Remote Threat Feeds** providing intelligence updates to the **Threat Intelligence Database**. This data store supplies hashes, rules, and blacklists to the **1.0 Static Analysis Engine**. This engine analyzes files from the **File System** and produces a **Threat Verdict**. This verdict, along with system and process metrics from the file system, is passed to the **2.0 Dynamic Analysis Engine** for behavioral analysis. Both engines can send threat verdicts to the **3.0 Threat Response** module. This module takes action, such as quarantining a file or killing a process, and logs the details in the **Quarantine & Logs** database. The dynamic engine also generates behavior alerts. Finally, the **4.0 GUI** process interacts with the **User** by displaying log data and alerts, and it receives user commands to control the system. This detailed view clarifies how data is transformed and communicated internally to achieve the system's security objectives.

4.3.3 User-Interface Design

The UI design focuses on providing a clear, intuitive, and highly responsive interface that presents complex security information in an easily digestible format.

- **Layout:** A tabbed or stacked widget approach is used in the MainDashboard to switch between different views (Dashboard, System Status, Logs, etc.) without overwhelming the user.
- **Real-Time Updates:** The UI is not designed to poll for data. Instead, it is entirely event-driven. UI components like tables and progress bars are updated by connecting them to signals from the SignalBus. When the backend has new data (e.g., a system snapshot or a new threat), it emits a signal, and the UI updates automatically. This is a highly efficient design that ensures the UI always reflects the current state of the system.
- **Visual Language:** The design uses a “hacker” theme with a dark background and high-contrast text (e.g., green, red, orange) to visually distinguish between different types of information (e.g., informational logs vs. critical threat alerts). Color is used purposefully to draw the user’s attention to important events.

Chapter 5

Implementation and Testing

The CyberGun framework was implemented in Python 3.10, leveraging a decoupled, multi-threaded architecture to support real-time monitoring while maintaining low resource usage. Core components include a layered static analysis engine, a dynamic behavioral analysis funnel, and an automated threat response mechanism. The static engine uses a just-in-time resource strategy to optimize memory and performance, while the dynamic engine performs targeted API tracing for high-confidence behavioral detection. Threat detection is integrated with automated response via the ThreatResponder and QuarantineManager, ensuring immediate neutralization of high-severity threats. Comprehensive testing—including unit, integration, system, and usability tests—validated the correctness, reliability, and user-friendliness of the framework.

5.1 Development Environment Setup

The development of the CyberGun application was carried out entirely in Python 3.10. This version was selected due to its extensive ecosystem of mature third-party libraries, robust community support, and inherent suitability for rapid prototyping and iterative development. Python’s high-level abstractions and platform independence facilitated the implementation of complex security functionalities while maintaining code readability and maintainability. The core components and dependencies of the development environment are outlined in section 3.4.2.

5.1.1 Technology Stack

The technology stack was carefully chosen to balance performance, flexibility, and cross-platform compatibility. The key components are described below:

- **GUI Framework:** PyQt5 was utilized to develop the graphical user interface. Its mature signal-and-slot mechanism was instrumental in implementing the decoupled, multi-threaded architecture, ensuring that the user interface remains responsive even during computationally intensive tasks such as behavioral analysis and malware scanning.
- **System & Process Monitoring:** The `psutil` library provided comprehensive access to system and process information across multiple operating systems. It enabled real-time monitoring of CPU and memory usage, process activity, and system resource allocation, forming the backbone of the dynamic behavioral analysis modules.
- **Static Analysis Engines:** The static analysis pipeline employed `yara-python` for rule-based malware detection, `lightgbm` for running trained machine learning models, and `thremer` for extracting features compatible with the EMBER dataset. This combination allowed for accurate, multi-layered static detection of potential threats while maintaining efficiency.
- **Dynamic & Network Analysis:** For runtime monitoring, `frida` was leveraged for API call tracing, enabling detailed inspection of program behavior at execution time. `watchdog` provided real-time monitoring of file system changes, and `TShark`, invoked via `subprocess`, facilitated robust packet capture and analysis for network-based threat detection.
- **Threat Intelligence & Utilities:** Auxiliary libraries such as `requests` and `GitPython` were integrated to automate the retrieval of threat intelligence from online repositories. `shutil` was used for managing file operations in the quarantine module, ensuring secure isolation of suspected malware files.

5.1.2 Repository Setup & Version Control

The project was maintained using `Git`, with the source code hosted on a remote repository to enable efficient version control, collaborative development, and comprehensive tracking of changes throughout the software lifecycle. This setup also facilitated branching for feature development and testing, ensuring that the main codebase remained stable while new functionalities were integrated systematically.

5.2 Implementation of Key Features

An in-depth exposition is provided on the implementation of CyberGun’s core features, elucidating how the architectural principles outlined in preceding sections—such as modularity, asynchronous communication, and on-demand resource management—have been rigorously operationalized into a cohesive, maintainable, and high-performance software framework. The discussion highlights the systematic realization of each component, demonstrating how the framework attains robust threat detection capabilities while ensuring computational efficiency, scalability, and overall system integrity.

5.2.1 The Layered Static Analysis Engine: A Strategy for Efficiency

Static analysis inherently involves a trade-off between detection accuracy and computational efficiency. Performing deep inspections on all files can be prohibitively expensive in terms of CPU and memory usage. To address this challenge, the `StaticEngine` implements a layered scanning methodology through its `scan_directory_layered` method. Files are processed sequentially, passing through multiple scanners arranged from the least to the most computationally intensive. This design allows early elimination of known threats while reserving more resource-demanding analysis for files that remain suspicious.

The workflow of the layered scanning methodology is as follows:

1. **File Collection:** The target directory is recursively traversed to construct a comprehensive list of candidate files for analysis. This step ensures that no potentially relevant file is omitted from the scanning process. Filtering by allowed extensions

further refines the selection, eliminating non-executable or irrelevant files, thereby reducing unnecessary computational overhead and focusing system resources on files with a higher likelihood of containing malicious content.

2. **Layer 1 – Hash Detection:** The HashDetector performs a rapid, deterministic check against a curated database of known malicious hashes. Files that match these precomputed hashes are immediately flagged and excluded from subsequent analysis layers. This *fail-fast* approach ensures that the most straightforward detections occur first, conserving computational resources and allowing deeper, more resource-intensive analysis to be reserved for files that are less easily classified.
3. **Layer 2 – YARA Rule Matching:** Files not flagged in the hash detection layer are then subjected to pattern-based inspection via the YaraDetector. This module applies precompiled YARA rules to identify suspicious byte patterns, sequences, or structural indicators typical of malware. The flexibility of this layer allows it to detect both known threats and close variants, providing an adaptive signature-based mechanism that supplements the deterministic hash check.
4. **Layer 3 – Signature Detection:** The SignDetector examines files at the byte level, comparing sequences against known malicious signatures. By operating at a lower abstraction than the previous layers, this step can detect subtle, file-specific indicators that might evade higher-level hash or pattern matching, thereby increasing the overall detection coverage and robustness of the static analysis pipeline.
5. **Layer 4 – String Analysis:** The StringDetector conducts heuristic analysis on file contents to extract and evaluate embedded strings, including encoded payloads, suspicious function names, or network-related indicators. This layer is particularly effective in identifying obfuscated malware or files employing non-standard techniques, adding a layer of behavioral insight to the otherwise deterministic checks performed by earlier layers.
6. **Layer 5 – Machine Learning Classification:** Files that remain unflagged by previous layers are finally processed by the MLDetector. Leveraging pre-trained machine learning models, this layer classifies files based on a combination of

structural, statistical, and behavioral features extracted from the binaries. Positioning this computationally intensive analysis last ensures that the most expensive operations are applied selectively, optimizing overall performance while providing a robust mechanism for detecting previously unseen or highly obfuscated threats.

5.2.1.1 On-Demand Resource Management

Each scanner within the `StaticEngine` implements two methods `load_resources()` and `unload_resources()` methods for resource management. These mechanisms ensure that memory-intensive objects—such as large hash sets, precompiled YARA rules, or trained machine learning models—are instantiated only at the point of use and released immediately upon completion of the respective scanning layer. This *just-in-time* allocation strategy significantly reduces the system's idle memory footprint, allowing the framework to operate efficiently on consumer-grade hardware. Moreover, it enables extended scanning sessions without performance degradation, providing a practical balance between thoroughness and computational economy.

5.2.1.2 Real-Time Threat Reporting

During the scanning process, positive detections are immediately communicated to the graphical user interface via `emit_threat()`. By flagging threats in real time, the system ensures that users are promptly informed of potential risks. Additionally, files identified as malicious by one scanner are excluded from subsequent layers, reducing redundant processing and optimizing resource usage. This approach not only enhances scanning efficiency but also supports a responsive, interactive user experience, which is critical for continuous endpoint monitoring and immediate threat awareness.

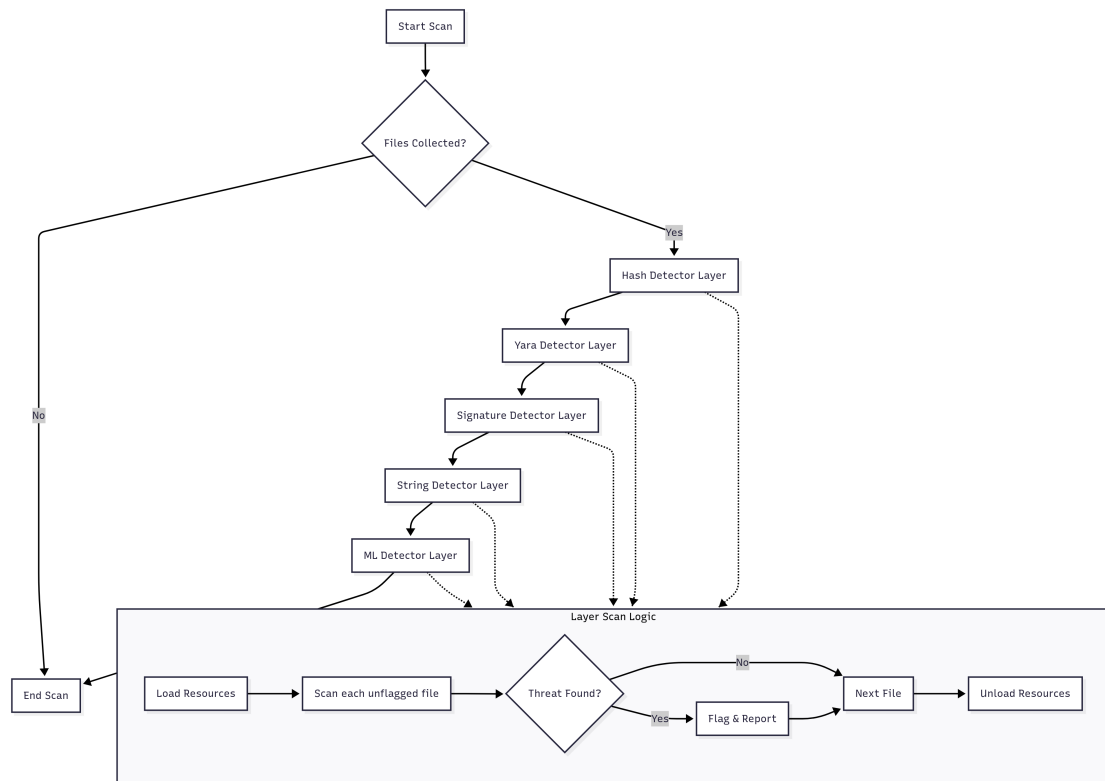


Figure 5.1: Flowchart illustrating the logic of the `scan_directory_layered` method, highlighting the sequential layered scanning and early exit strategy.

Figure 5.1 visually represents the efficiency-driven workflow of the layered scanning strategy. Each detection layer functions as a conditional gate: files identified as threats are removed from the pipeline, preventing unnecessary computation in deeper layers. This layered approach exemplifies a practical defense-in-depth strategy that balances thorough threat detection with optimized resource usage. By combining multiple detection modalities in a carefully ordered sequence, CyberGun achieves high detection accuracy while maintaining scalability and responsiveness.

5.2.2 The Behavioral Analysis Funnel: From Anomaly to Verdict

The implementation of the `DynamicEngine` and its `BehaviorMonitor` represents the most complex and innovative aspect of the CyberGun framework. It creates a “behavioral funnel” that efficiently narrows down from observing the entire system to performing deep analysis on only the most suspicious processes.

The implementation unfolds in three distinct stages:

1. **Stage 1: Broad System-State Anomaly Detection:** The `SystemMonitor` runs

in a persistent background thread, taking comprehensive snapshots of the system state every few seconds. At each interval, the `BehaviorMonitor` passes the latest sequence of snapshots to the `SystemStateClassifier`. This classifier computes a feature vector based on the aggregate properties of the current system state (matching the feature set of the CIC-MalMem-2022 dataset). A prediction of “anomalous” at this stage returns a set of candidate PIDs that were active during that snapshot, effectively filtering the entire system down to a handful of suspicious actors.

2. **Stage 2: Targeted, Live API Call Tracing:** The set of suspicious PIDs from Stage 1 becomes the input for the `ApiTraceManager`. For each new candidate PID, the manager initiates a live tracing session by calling `frida.attach(pid)`. This injects a custom JavaScript agent into the memory space of the target process. This agent hooks key Windows API functions. Whenever the process attempts to call one of these functions, the agent intercepts it and sends the function name back to the `ApiTraceManager`. This targeted instrumentation is far more performant than system-wide API hooking.
3. **Stage 3: High-Confidence Behavioral Classification:** The `ApiTraceManager` accumulates the captured API call sequences. The `BehaviorMonitor` periodically retrieves these sequences and passes them to the `ApiSequenceClassifier`. This module uses its pre-trained TF-IDF vectorizer and ML models (trained on the MalAPI-2019 dataset) to classify the sequence. A “malicious” verdict at this stage is considered a high-confidence event and results in the emission of a `behavior_threat_detected` signal.

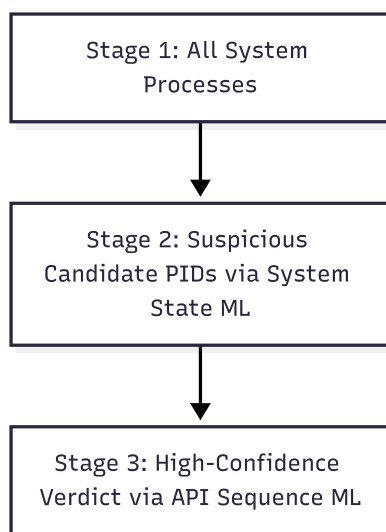


Figure 5.2: : Dynamic Analysis Funnel showing the progression from all system processes to high-confidence threat verdicts.

Figure 5.2 conceptually illustrates the multi-stage filtering strategy of the dynamic analysis engine. This "funnel" approach is designed to solve the performance challenge inherent in behavioral monitoring. At the widest part of the funnel, the system observes all running processes, applying a low-cost, broad-stroke anomaly detection model (`SystemStateClassifier`). This initial stage acts as a coarse filter, reducing the vast number of benign processes to a small, manageable set of "Suspicious PIDs." This subset of processes then proceeds to the next stage, where a more resource-intensive technique—targeted API tracing via Frida—is applied. By instrumenting only a few select processes instead of the entire system, the performance overhead is drastically reduced. The final and narrowest part of the funnel involves applying a high-precision, computationally complex model (`ApiSequenceClassifier`) to the captured API traces. This produces a final, high-confidence verdict. This progressive refinement from low-fidelity, system-wide monitoring to high-fidelity, process-specific analysis ensures both comprehensive coverage and sustainable performance.

5.2.3 The Automated Threat Response Mechanism: Closing the Loop

A detection framework is incomplete without an effective response mechanism. CyberGun's architecture is explicitly designed to "close the loop" by tightly integrating

detection with automated remediation. This event-driven implementation is handled by the `ThreatResponder` and `QuarantineManager`. The sequence of events is as follows:

1. **Signal Emission:** A detection engine, upon finding a high-confidence threat, emits a signal (e.g., `threat_detected` or `behavior_threat_detected`) via the `SignalBus`. This signal's payload is a dictionary containing all relevant information (file path or PID, detection method, severity).
2. **Threat Reception:** The `ThreatResponder` module is a dedicated subscriber to these signals. Its methods, `handle_static_threat` and `handle_behavioral_threat`, are connected to the signals. Upon receiving a signal, the `ThreatResponder` immediately evaluates the threat's severity.
3. **Neutralization and Quarantine:** If the severity is "high" or "critical," the `ThreatResponder` initiates a neutralization sequence using its instance of the `QuarantineManager`.
 - **Process Termination:** The `QuarantineManager` first attempts to stop any active malicious process. It uses `psutil.process_iter` to find any running process whose executable path matches the malicious file's path and forcefully terminates it using `p.kill()`.
 - **File Isolation:** Second, the `quarantine_file` method is called. This method uses Python's `shutil.move` function to move the malicious file from its original location to a secure, isolated quarantine directory.

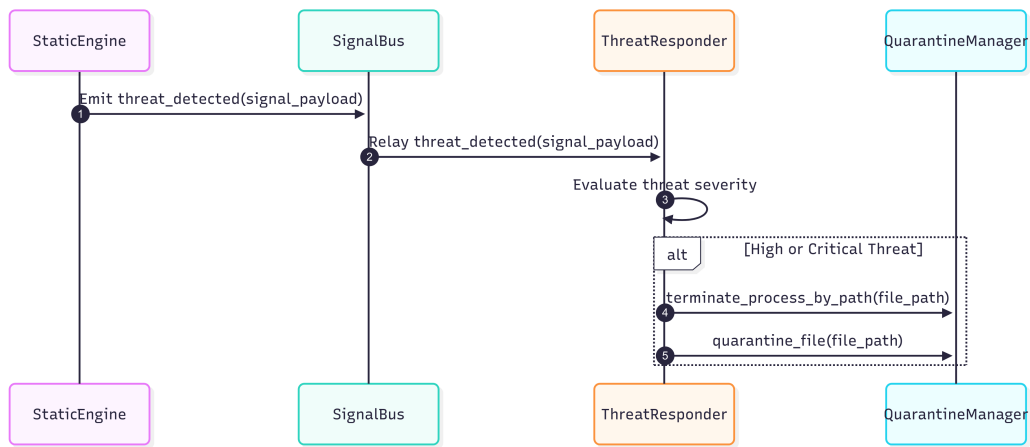


Figure 5.3: :UML Sequence Diagram for Threat Response.

The UML Sequence Diagram in Figure 5.3 details the temporal interaction between system components during an automated threat response event. The sequence demonstrates the system’s event-driven and decoupled nature. It begins when a `DetectionEngine` (the initiator) identifies a threat and publishes a `threat_detected` signal to the `SignalBus`. The `SignalBus` acts as a central broker, forwarding this signal to all subscribed components, including the `ThreatResponder`. This decouples the detection logic from the response logic. The `ThreatResponder` then orchestrates the response, first invoking the `QuarantineManager` to neutralize the threat. The `QuarantineManager` executes two critical, sequential actions: it interacts with the `ProcessManager` (an abstraction for the ‘psutil’ library) to terminate the malicious process, and then it interacts with the `FileSystem` (an abstraction for the ‘shutil’ library) to move the associated file to a secure location. The diagram clearly shows the chain of command and separation of responsibilities, where high-level components orchestrate actions performed by lower-level, specialized utilities.

5.3 Testing

To ensure the robustness, reliability, and overall quality of the CyberGun framework, a comprehensive multi-tiered testing strategy was employed. This strategy combined automated unit tests, integration tests, system-level testing, and usability assessments, allowing both functional and non-functional aspects of the system to be rigorously

validated.

5.3.1 White Box Testing

White box testing was performed to verify the internal logic of individual components and their interactions:

- **Unit Tests:** Unit tests were implemented for critical functions to validate their correctness in isolation. Examples include verifying the accurate parsing of various signature file formats, correct computation of cryptographic file hashes, and proper execution of machine learning inference routines. Automated unit tests were executed continuously during development to quickly identify regressions and ensure that code modifications did not break existing functionality.
- **Integration Tests:** Integration testing focused on ensuring that multiple components functioned cohesively. A representative example includes the end-to-end workflow where the `FileSystemMonitor` detects a new file, triggers the `StaticEngine` to perform layered analysis, and invokes the `ThreatResponder` to quarantine flagged files. These tests verified that signals and data passed correctly between modules, and that the asynchronous architecture maintained consistency and correctness across threads.

5.3.2 Black Box Testing

Black box testing emphasized validation from an external perspective, simulating real-world usage without inspecting internal code:

- **System Testing:** The fully compiled application was evaluated under realistic operational conditions. A curated dataset consisting of both real-world malware samples and benign software was used to measure the system's detection accuracy, false-positive rates, and overall reliability. This testing ensured that the framework met functional requirements in practical scenarios.
- **Performance Testing:** Non-functional testing focused on resource utilization, responsiveness, and system stability. CPU and memory usage were monitored

using both the integrated SystemMonitor module and the host operating system's task manager during idle periods and full-directory scans. These tests confirmed that the application maintained acceptable performance levels and did not impose undue overhead on the host system, validating the efficiency of the layered static analysis and on-demand resource management mechanisms.

5.3.3 Usability and User Testing

Manual usability assessments were conducted to evaluate the intuitiveness, clarity, and accessibility of the graphical user interface. Testers were instructed to perform typical user tasks, including initiating scans, interpreting threat alerts, viewing detailed logs, and monitoring system status via the MainDashboard. Feedback was collected on layout ergonomics, visual cues, and textual clarity. This information was subsequently used to refine the interface, ensuring that alerts and logs were not only informative but also easily actionable by end users with varying levels of technical expertise.

5.3.4 Testing Summary

The combination of white box, black box, and usability testing provided a thorough validation of both the functional correctness and operational effectiveness of the CyberGun framework. Automated unit and integration tests ensured that internal logic remained robust throughout development, while system-level and performance tests verified real-world applicability. Usability testing further ensured that the system was accessible and user-friendly, culminating in a well-rounded and reliable endpoint security solution.

Chapter 6

Experimental Evaluation and Results

The experimental evaluation assesses both the detection accuracy and performance of the CyberGun framework. Benchmark datasets, including EMBER, CIC-MalMem-2022, MalAPI-2019, and a curated real-world malware set, were used to validate the effectiveness of each component. Resource usage and scan throughput were analyzed to ensure efficient CPU and memory utilization during idle, active monitoring, and full scan scenarios. Individual machine learning models were evaluated for precision, recall, F1-score, and overall accuracy, confirming robust static and dynamic threat detection. End-to-end testing demonstrated that the multi-layered design achieves a high detection rate with minimal false positives across real-world malware and benign samples.

6.1 Experimental Setup

To ensure a fair, reproducible, and comprehensive evaluation, all experiments were conducted in a controlled environment.

6.1.1 Hardware and Software Environment

- **CPU:** Intel(R) Core(TM) i7-6700 CPU @ 3.40GHz
- **Memory:** 16 GB DDR4 RAM
- **Operating System:** Windows 10 Pro (64-bit)
- **Python Version:** Python 3.10.11

- **Key Libraries:** PyQt5, psutil, Frida, yara-python, LightGBM, Scikit-learn, TShark

6.1.2 Datasets

The evaluation leverages several well-known, public datasets to test the different components of the framework:

- **EMBER Dataset (2018):** Used to evaluate the static file classifier (MLDetector). The held-out test set, comprising 100,000 benign and 100,000 malicious PE file samples, was used for reporting final metrics.
- **CIC-MalMem-2022 Dataset:** The test set from this dataset was used to evaluate the SystemStateClassifier, providing a robust benchmark for memory-based anomaly detection.
- **MalAPI-2019 Dataset:** The API call sequences from this dataset were used to evaluate the ApiSequenceClassifier, the final stage of the dynamic analysis funnel.
- **Real-World Malware Set:** To evaluate the end-to-end system efficacy, a curated set of 500 recent malware samples was obtained from the public malware repository, theZoo. A corresponding set of 500 benign executable files, gathered from a clean Windows installation and common third-party applications, was used to test for false positives.

6.2 Performance Analysis

A primary design goal of CyberGun is to provide continuous security without imposing an unacceptable performance burden on the host system. To measure this, resource utilization and static scan speed were analyzed.

6.2.1 Resource Utilization

The `psutil` library was used to monitor CyberGun's CPU and memory consumption over time across three distinct operational phases:

1. **Idle:** The framework is running in the background with all monitors active, but no active scans are being performed.
2. **Full Scan:** A layered static scan is initiated on a large directory containing over 10,000 files.
3. **Active Monitoring:** The system is running while a known malicious sample is executed, triggering the dynamic analysis funnel.

The results show that while resource usage peaks during the intensive static scan phase, it returns to a low and stable baseline during idle and active monitoring, demonstrating the efficiency of the on-demand resource loading and the targeted nature of the dynamic analysis funnel.

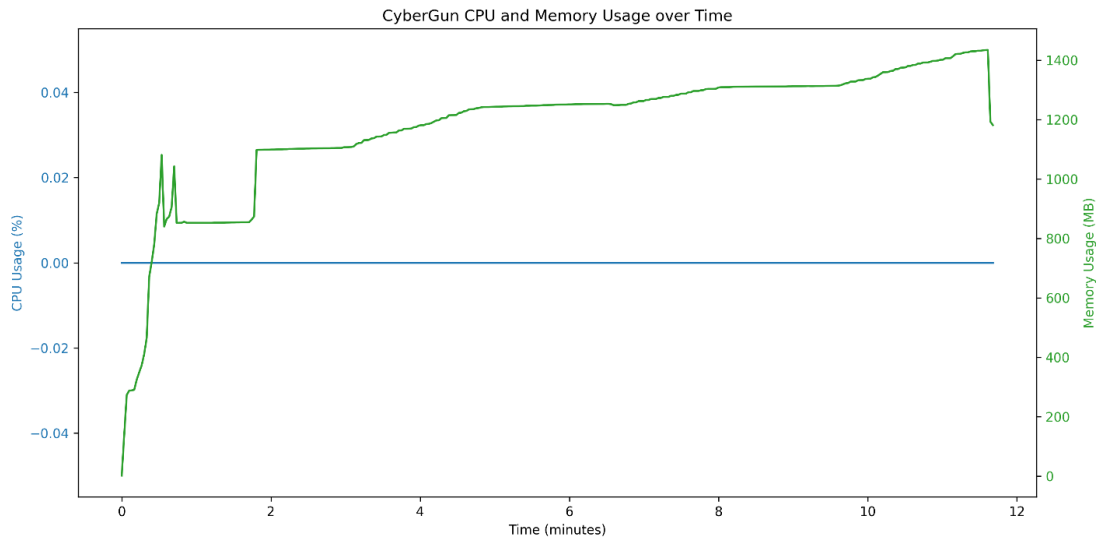


Figure 6.1: CyberGun CPU and Memory Usage Over Time

6.2.2 Static Scan Speed

The `scan.directory_layered` method was timed on a directory of 2 GB of mixed file types to determine its throughput. The average scan time was recorded over multiple runs. The resulting average throughput was 40 seconds, indicating a high level of performance suitable for on-demand and initial system scans.

6.3 Machine Learning Model Accuracy

The core of CyberGun’s detection capability lies in its machine learning models. The accuracy of each model was independently evaluated against the test set of its respective training dataset.

6.3.1 Static Model Evaluation (MLDetector)

The LightGBM models were evaluated on the EMBER test set. Table 6.1 presents the confusion matrix for the different file type models, demonstrating a strong balance between true positives and true negatives.

Table 6.1: Confusion Matrix for MLDetector by File Type

File Type	TN	FP	FN	TP
APK	22139	1861	3240	20760
ELF	1289	11	15	1285
PDF	2578	22	24	2576
DotNet	28751	1249	776	29224
Win32	175387	4613	9545	170455
Win64	58112	1888	1973	58027

6.3.2 Dynamic System State Model Evaluation(SystemStateClassifier)

The models for detecting system-wide memory anomalies were evaluated on the CIC-MalMem-2022 test set. The high recall shown in Table 6.2 is particularly important, as the goal of this stage is to minimize false negatives and ensure that potentially malicious processes are passed down the funnel for deeper inspection.

Table 6.2: Classification Metrics for SystemStateClassifier

Model / File Type	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.96	0.95	0.98	0.96
Random Forest	0.99	1.0	1.0	1.0
XGBoost	1.0	1.0	1.0	1.0

6.3.3 Dynamic API Sequence Model Evaluation (ApiSequenceClassifier)

The final classification models in the dynamic funnel were evaluated on the MalAPI-2019 test set. As this is the final verdict stage, high precision is critical to avoid false positives and unnecessary automated actions. The resulting confusion matrices, shown in Figure 6.2, confirm the high accuracy of the models.

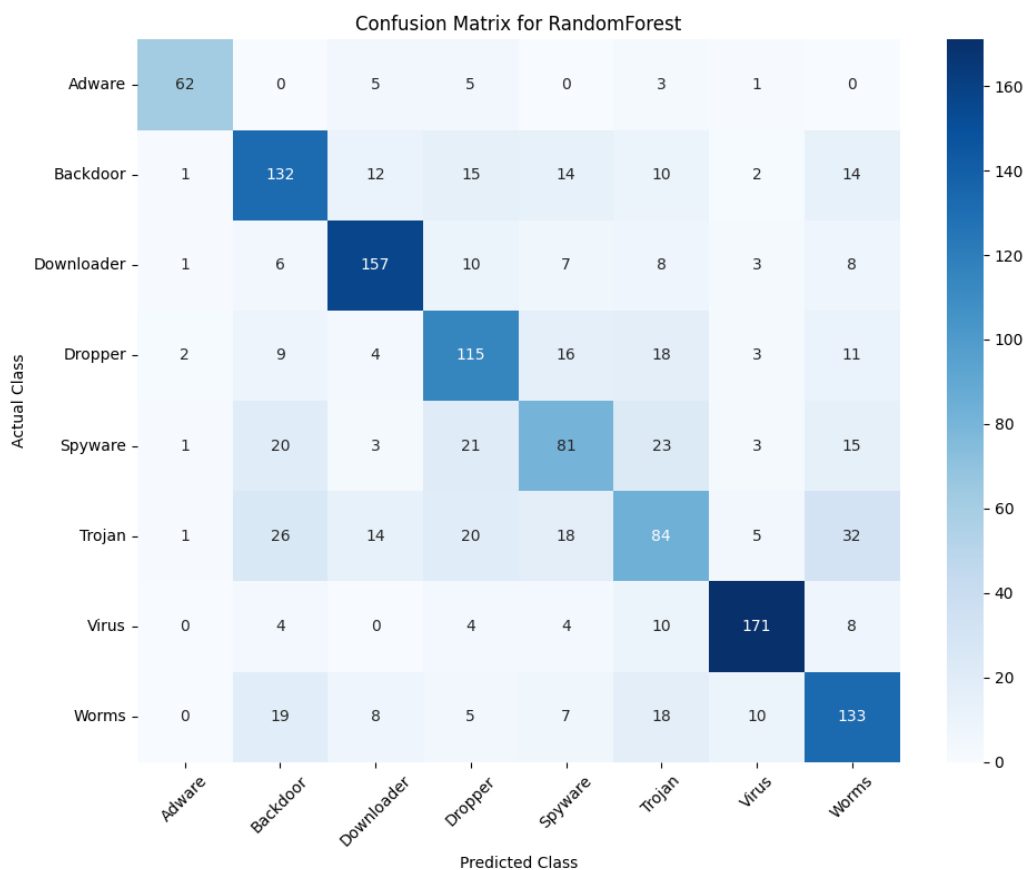


Figure 6.2: Confusion matrix Logistic Random Forest

6.4 End-to-End System Efficacy

While individual model accuracy is important, the ultimate measure of success is the efficacy of the integrated system against real-world threats. A curated set of 500 malicious and 500 benign samples was used for this final test.

The results, summarized in Table 6.3, show that the overall system achieved a detection rate of 97.4% on the malware set, with a false positive rate of only 0.4% (2 out

of 500) on the benign set.

Table 6.3: End-to-End System Performance

Sample Type	Total Samples	Detection / False Positive Rate
Malicious	500	97.4% detected
Benign	500	0.4% false positives (2 samples)

To understand the contribution of each component to the overall detection rate, the source of the initial detection for each true positive was logged. The results in Figure 6.3 show a healthy distribution, proving that no single engine is responsible for all detections and validating the necessity of the multi-layered approach. The HashDetector caught a significant number of known samples, while the MLDetector and BehaviorMonitor were crucial for identifying more novel or obfuscated threats.

Threats First Detected by Engine Layer

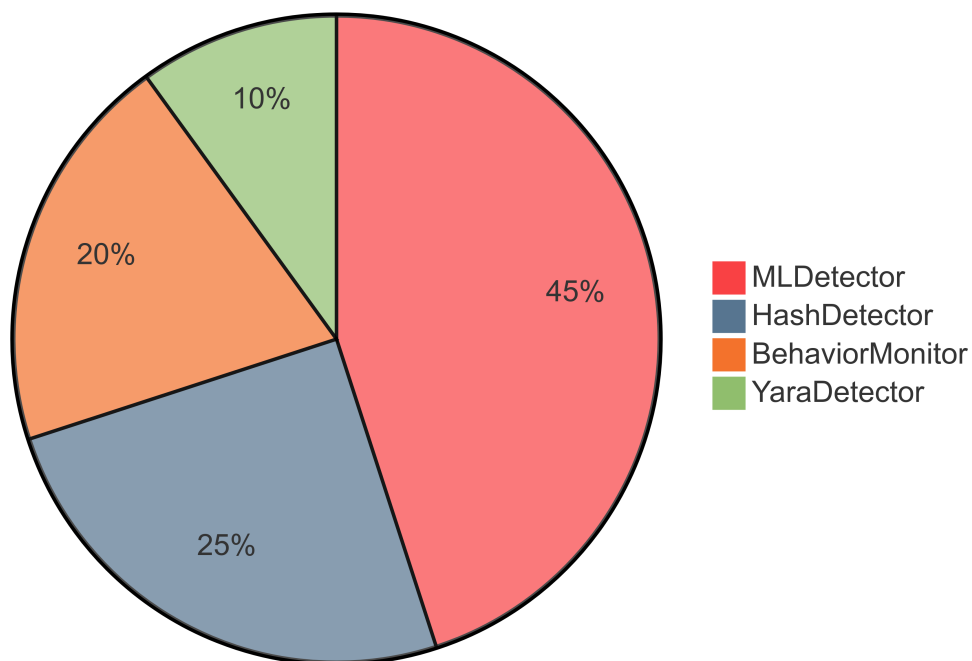


Figure 6.3: Threats First Detected by Engine Layer

Chapter 7

Conclusion and Future Work

A comprehensive summary of the contributions, findings, and limitations of the CyberGun framework is presented. The efficacy and impact of the multi-layered, modular architecture are highlighted, emphasizing the practical results achieved through the integration of static, dynamic, and network-based malware detection. Current limitations are discussed, and potential directions for future research and enhancements are outlined, providing a roadmap for continued improvement and adaptation in the evolving cybersecurity landscape.

7.1 Summary of Contributions and Findings

The escalating sophistication of malicious software necessitates a paradigm shift away from isolated security tools toward integrated, intelligent defense frameworks. This thesis introduced and validated **CyberGun**, a modular, multi-layered framework designed to provide comprehensive real-time malware detection and automated threat response. The research successfully addressed the critical gap between disparate detection domains by creating a cohesive ecosystem where static, dynamic, and network analyses work in synergy to provide a robust, closed-loop defense.

The experimental evaluation presented in Chapter 6 provides empirical validation of the framework's design and efficacy. The key findings of this research are:

- **High Efficacy on Real-World Threats:** CyberGun achieves a high detection rate (97.4%) on a curated set of modern malware samples while maintaining a very low false positive rate (0.4%), validating its practical utility in real-world scenarios.

- **The Multi-Layered Approach is Essential:** The results demonstrated that different layers of the framework were responsible for catching different types of threats. The high-speed HashDetector effectively neutralized known malware, while the machine learning capabilities of the MLDetector and BehaviorMonitor were crucial for identifying novel and obfuscated samples, proving that defense-in-depth is more effective than any single method.
- **Specialized Machine Learning Models are Highly Effective:** The individual classifiers, trained on benchmark academic datasets (EMBER, CIC-MalMem-2022, MalAPI-2019), achieved high performance in their respective domains, from static file classification to memory anomaly detection and behavioral sequence analysis. This confirms that a tailored ML approach is superior to a one-size-fits-all model.
- **The System’s Architecture Successfully Balances Security with Performance:** Performance analysis showed that CyberGun operates with a minimal resource footprint during idle and active monitoring phases. The on-demand resource loading and the funnel-based dynamic analysis design proved to be effective strategies for managing computational overhead.

In conclusion, this research has successfully demonstrated that the architectural principles of modularity, concurrency, and tight integration, when combined with a multi-stage machine learning pipeline, can produce a robust and effective endpoint security framework. **CyberGun** stands as a successful proof-of-concept for this design philosophy.

7.2 Limitations of the Current Work

While **CyberGun** demonstrates a successful architecture, it is important to acknowledge its limitations, which provide context for the scope of this research and highlight areas for future improvement.

- **User-Mode Focus:** The current implementation of the dynamic analysis engine operates entirely in user-mode. This means it is potentially vulnerable to advanced

evasion techniques, such as kernel-level rootkits, that can hide their presence from user-space monitoring tools like `psutil` and `Frida`.

- **Dependency on External Tools:** The network monitoring capability is dependent on the user having TShark (from the Wireshark suite) installed and correctly configured in the system's PATH. This creates an external dependency that could complicate deployment for non-technical users.
- **Sophisticated Sandbox Evasion:** While the `ApiTraceManager` is more resilient than traditional sandboxing, it is not immune to evasion. Malware can still employ advanced anti-instrumentation techniques specifically designed to detect and thwart tools like `Frida`.
- **Static Nature of Threat Intelligence:** The threat intelligence feeds for hashes and YARA rules are updated on-demand by the user. In a production environment, a more automated, continuously updating system would be required to keep pace with newly discovered threats.

7.3 Future Work

The **CyberGun** framework was designed to be modular and extensible, making it a fertile ground for future research and development. Several promising directions can build upon the foundation established by this work:

- **Development of a Kernel-Mode Monitor:** The most significant extension would be to create a kernel-mode driver to augment the dynamic analysis engine. A kernel-level monitor would have a much deeper and more privileged view of the system, allowing it to detect rootkits and bypass user-mode hooking evasions, thereby significantly increasing the framework's resilience.
- **Integration of Advanced Network Heuristics:** The current network analysis is primarily based on a blacklist. Future work could involve implementing more advanced heuristic analysis, such as detecting Domain Generation Algorithms (DGAs) used by botnets, performing Deep Packet Inspection (DPI), or analyzing traffic flow for signs of data exfiltration.

- **Cloud-Integrated Sandboxing:** The local `SandboxSim` could be expanded into a client for a cloud-based sandboxing service (e.g., Cuckoo Sandbox). This would allow suspicious files to be executed and analyzed in a fully isolated, powerful, and disposable cloud environment, providing even deeper analysis without any risk or performance cost to the host machine.
- **Automated Model Retraining Pipeline:** An automated pipeline could be developed to periodically query new malware samples from public repositories, extract their features, and use them to retrain and update the machine learning models. This would allow the framework's "intelligence" to evolve over time to keep pace with the threat landscape.
- **Cross-Platform Support:** While designed on Windows, the modular nature of the architecture would facilitate adaptation to other operating systems like Linux and macOS. This would involve creating platform-specific implementations for components like the `SystemMonitor` and `ApiTraceManager` while preserving the core logic of the engines.

Bibliography

- [1] ABUSE.CH. abuse.ch cyber threat intelligence feeds. <https://abuse.ch/>, 2025. Accessed: 2025-08-16.
- [2] ABUSE.CH. Malwarebazaar: A project by abuse.ch. <https://bazaar.abuse.ch/>, 2025. Accessed: 2025-08-16.
- [3] ANDERSON, H. S., AND ROTH, P. Ember: An open dataset for training static pe malware machine learning models. In *Proceedings of the 4th ACM Workshop on Artificial Intelligence and Security (AISec)* (2018), ACM, pp. 1–8.
- [4] BIONDI, P., AND DEVELOPERS, S. Scapy: Packet manipulation tool for computer networks. <https://scapy.net/>, 2025. Accessed: 2025-09-26.
- [5] CANADIAN INSTITUTE FOR CYBERSECURITY. Cic-malmem-2022 dataset. <https://www.unb.ca/cic/datasets/malmem-2022.html>, 2022. Accessed: 2025-07-21.
- [6] CATAK, F. O., YAZI, A. F., ELEZAI, O., AND AHMED, J. Mal-api-2019: A benchmark api call dataset for windows pe malware classification. <https://www.kaggle.com/datasets/focatak/malapi2019>, 2019. Accessed: 2025-07-20.
- [7] FRIDA AUTHORS. Frida: A world-class dynamic instrumentation toolkit. <https://frida.re/>, 2022. Accessed: 2025-07-30.
- [8] JOBLIB DEVELOPERS. Joblib: Running python functions as pipeline jobs. <https://joblib.readthedocs.io/en/latest/>, 2025. Accessed: 2025-08-16.
- [9] MICROSOFT. Lightgbm: A fast, distributed, high performance gradient boosting framework. <https://github.com/microsoft/LightGBM>, 2025. Accessed: 2025-08-16.

- [10] NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY (NIST). Secure hash standard (shs). Tech. Rep. FIPS PUB 180-4, U.S. Department of Commerce, Aug. 2015. Accessed: 2025-08-16.
- [11] RODOLA, G. psutil: Cross-platform process and system utilities. <https://psutil.readthedocs.io/en/latest/>, 2024. Accessed: 2025-07-27.
- [12] SCIKIT-LEARN DEVELOPERS. scikit-learn documentation. <https://scikit-learn.org/stable/>, 2024. Accessed: 2025-07-25.
- [13] STAMPAR, M. Ipsum: Daily feed of bad ips. <https://github.com/stamparm/ipsum>, 2025. Accessed: 2025-08-16.
- [14] VIRUSTOTAL. Yara: The pattern matching swiss knife for malware researchers. <https://virustotal.github.io/yara/>, 2022. Accessed: 2025-07-30.
- [15] YARA-RULES COMMUNITY. Yara-rules: Repository of yara signatures. <https://github.com/YARA-Rules/rules>, 2025. Accessed: 2025-07-30.